

OPTIMIZATION AND REGULARIZATION

OSCAR LLORENTE GONZALEZ

OLLLORENTE@COMILLAS.EDU

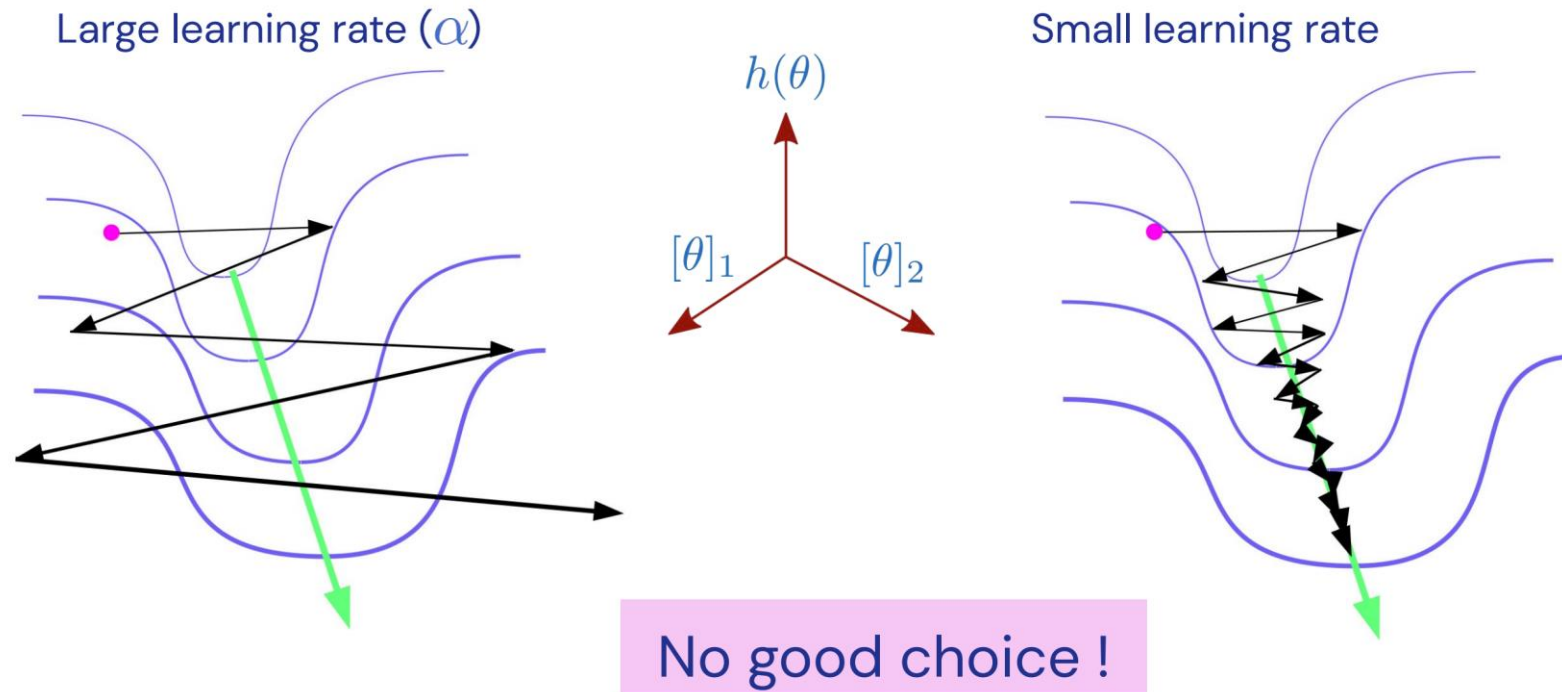
INDEX

- Optimization algorithms
- Overfitting and underfitting
- Normalization
- Data Augmentation
- Early stopping
- Dropout
- Residual Connections
- Learning rate schedulers
- Regularization
- Ensembles
- Transfer Learning



1 - OPTIMIZATION ALGORITHMS

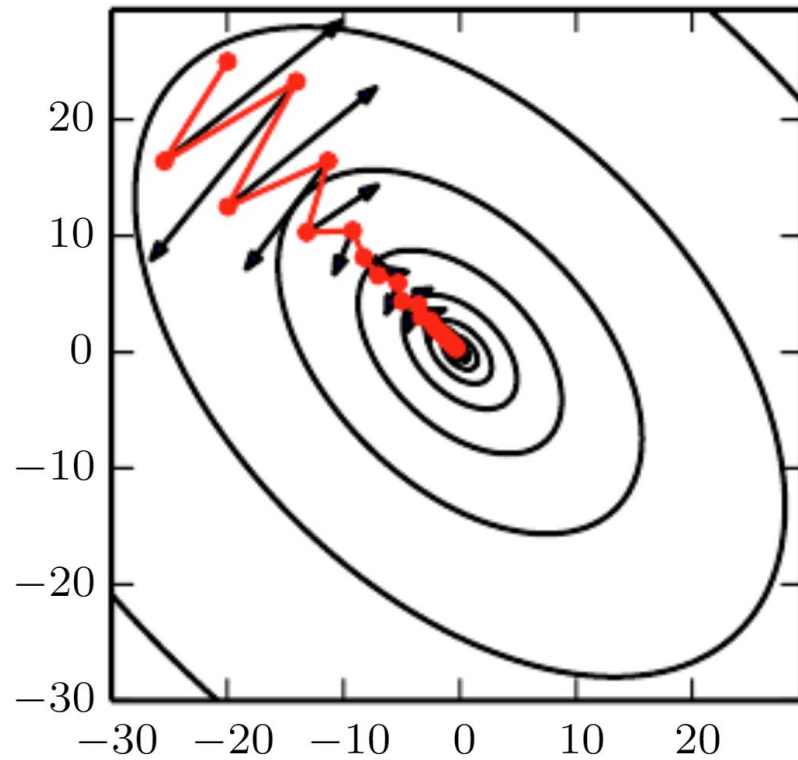
THE PROBLEM WITH SGD



The diagram shows a blue curve with several local minima. A green dot is placed on one of the upper parts of the curve. From this dot, a green arrow points downwards and to the right, following the slope of the curve, representing the direction of the negative gradient towards a local minimum.



SGD MOMENTUM



$$\mathbf{w}_{t+1} = \mathbf{w}_t + \mathbf{v}_{t+1}$$

$$\mathbf{v}_{t+1} = \rho \mathbf{v}_t - \eta \mathbf{g}(\mathbf{w}_t)$$

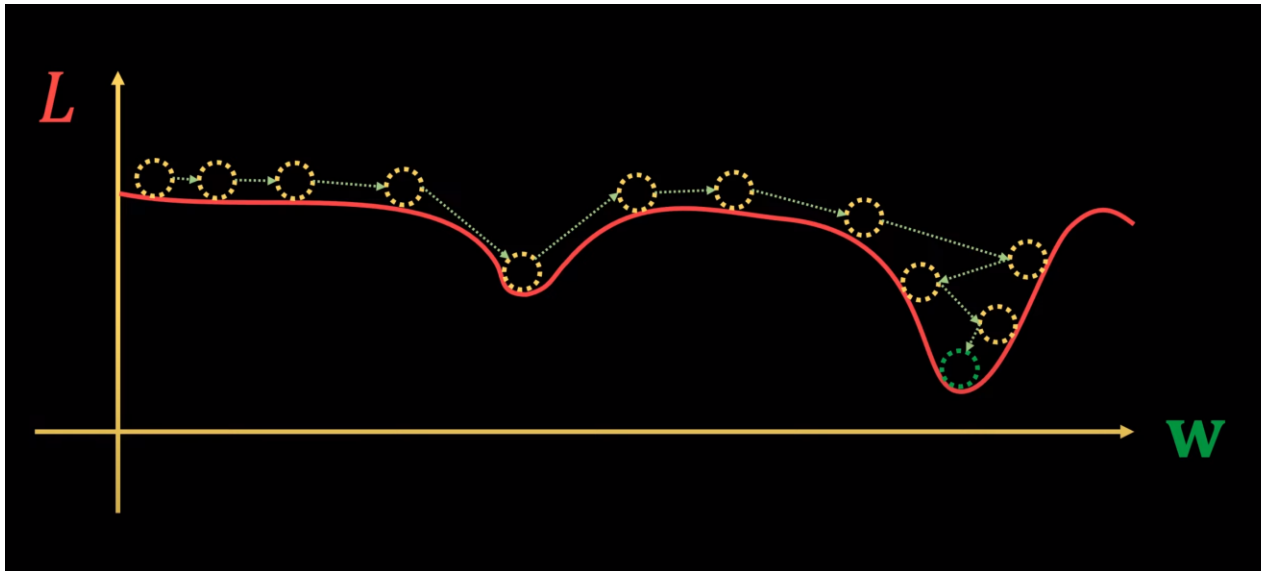
Update rule

Velocity

Momentum



SGD MOMENTUM

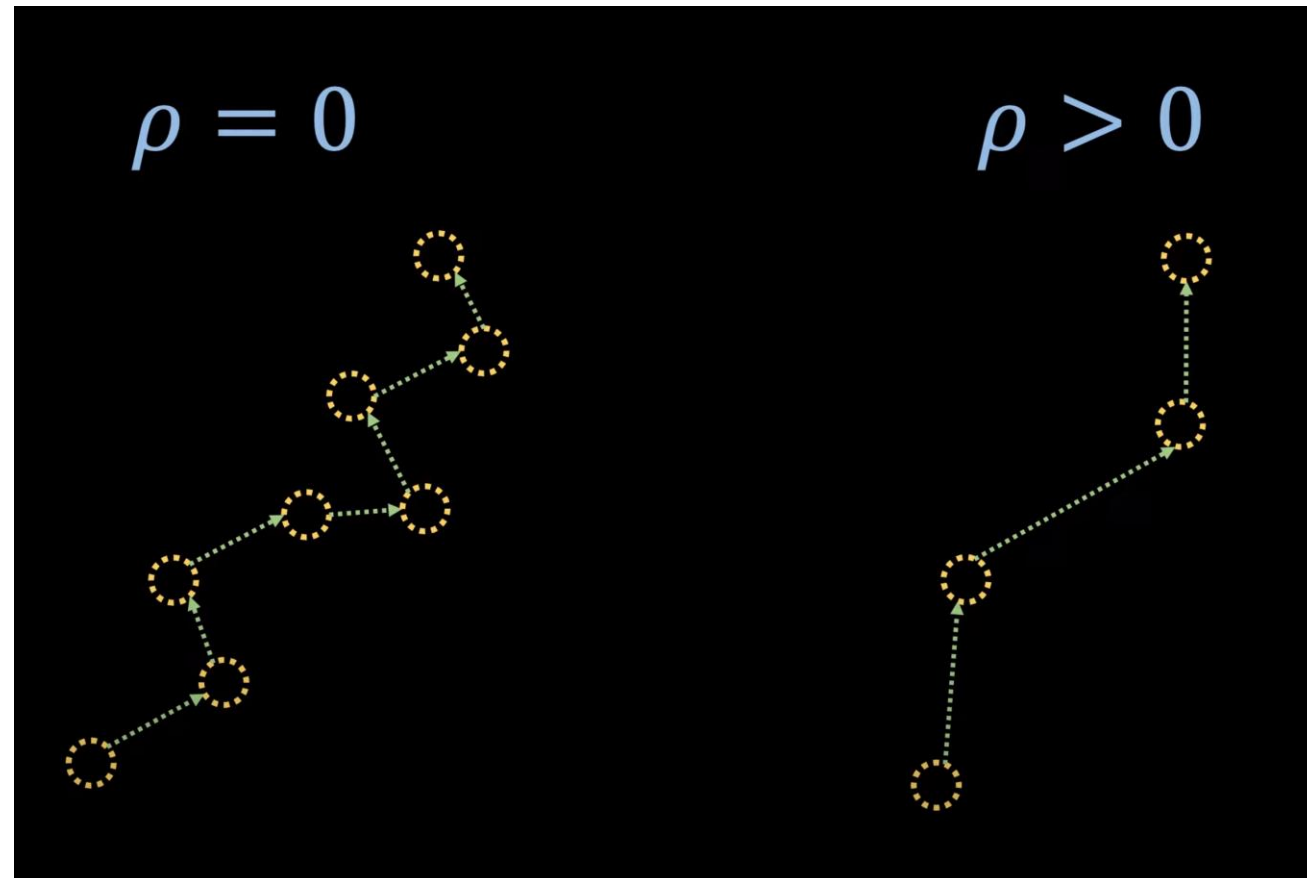


$$\mathbf{w}_{t+1} = \mathbf{w}_t + \overbrace{\rho \mathbf{v}_t}^{\text{Velocity jump}} - \overbrace{\eta \mathbf{g}(\mathbf{w}_t)}^{\text{Gradient jump}}$$

Update rule



SGD MOMENTUM TRAJECTORY



CLASSICAL VS NESTEROV MOMENTUM

“Classical” momentum

$$\mathbf{v}_{t+1} = \rho \mathbf{v}_t - \eta \mathbf{g}(\mathbf{w}_t)$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \mathbf{v}_{t+1}$$

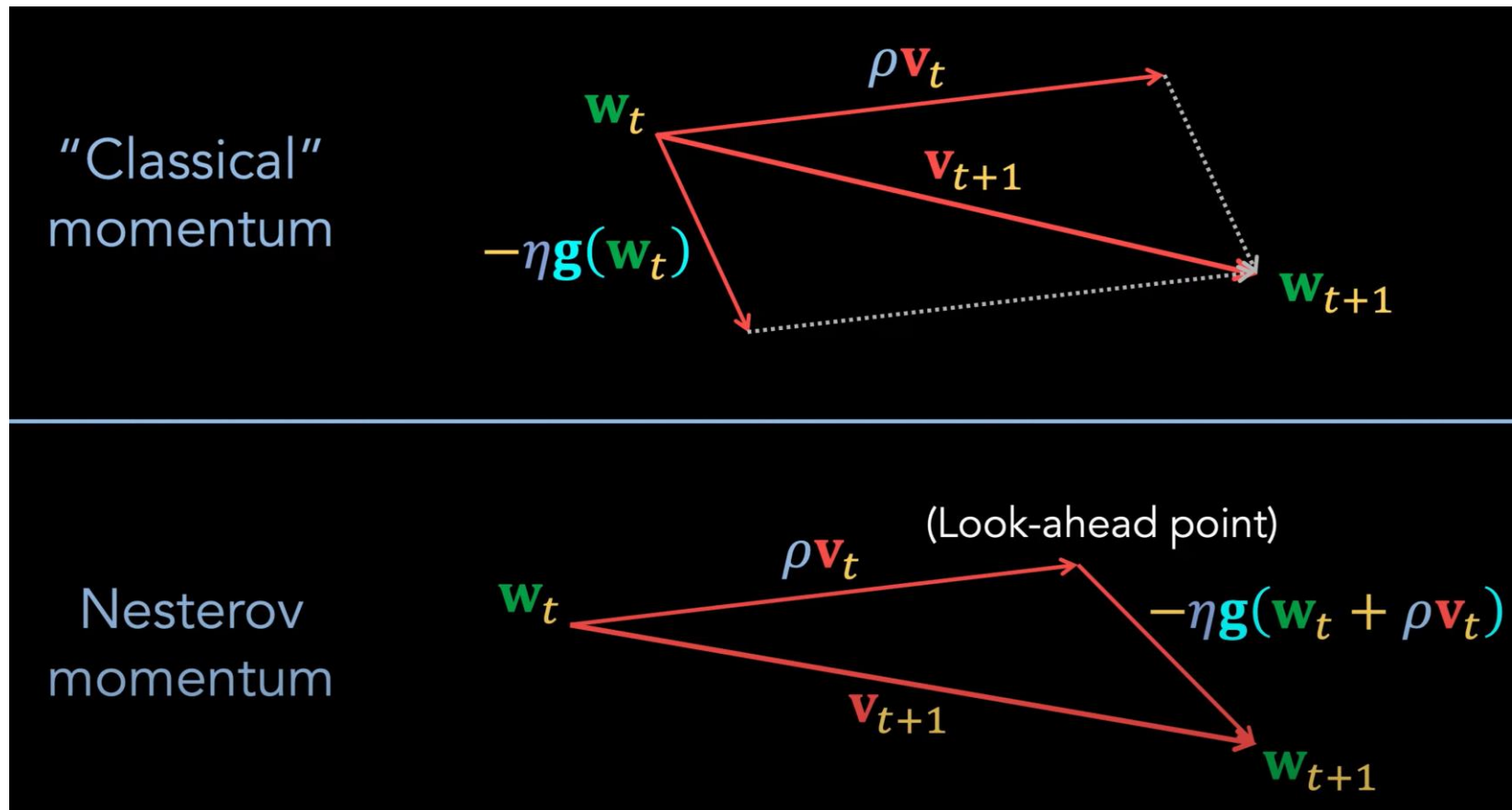
Nesterov momentum

$$\mathbf{v}_{t+1} = \rho \mathbf{v}_t - \eta \mathbf{g}(\mathbf{w}_t + \rho \mathbf{v}_t)$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \mathbf{v}_{t+1}$$



CLASSICAL VS NESTEROV MOMENTUM



ADAGRAD

$$\omega_{i,t+1} = \omega_{i,t} - \frac{\eta}{\varepsilon + \sqrt{\sum_{j=1}^t g(\omega_{i,j})^2}} g(\omega_{i,t})$$

Update rule



ADAGRAD

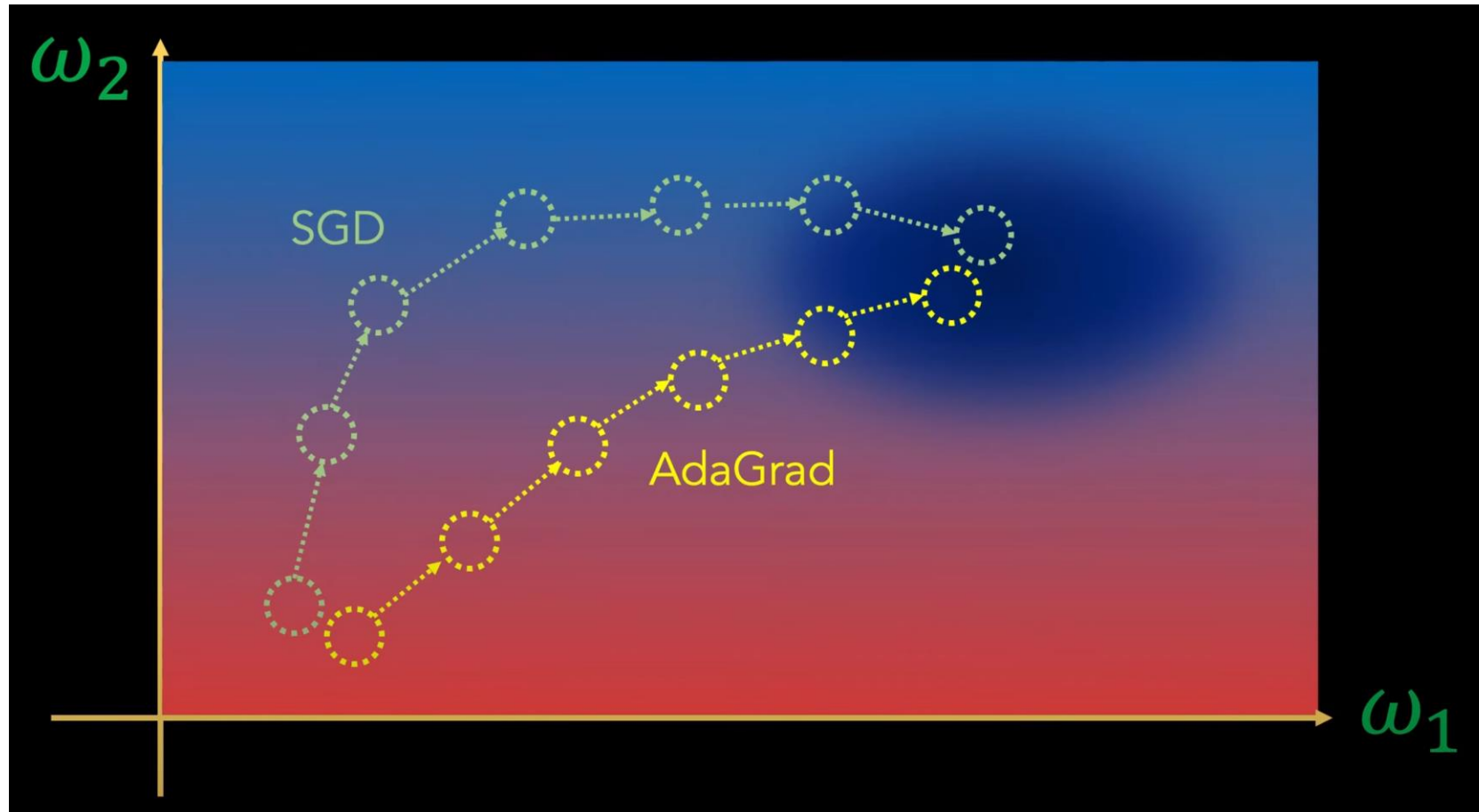
$$\mathbf{v}_{t+1} = \mathbf{v}_t + \mathbf{g}(\mathbf{w}_t)^2$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \frac{\eta}{\epsilon + \sqrt{\mathbf{v}_{t+1}}} \mathbf{g}(\mathbf{w}_t)$$

Update rule



ADAGRAD VS SGD



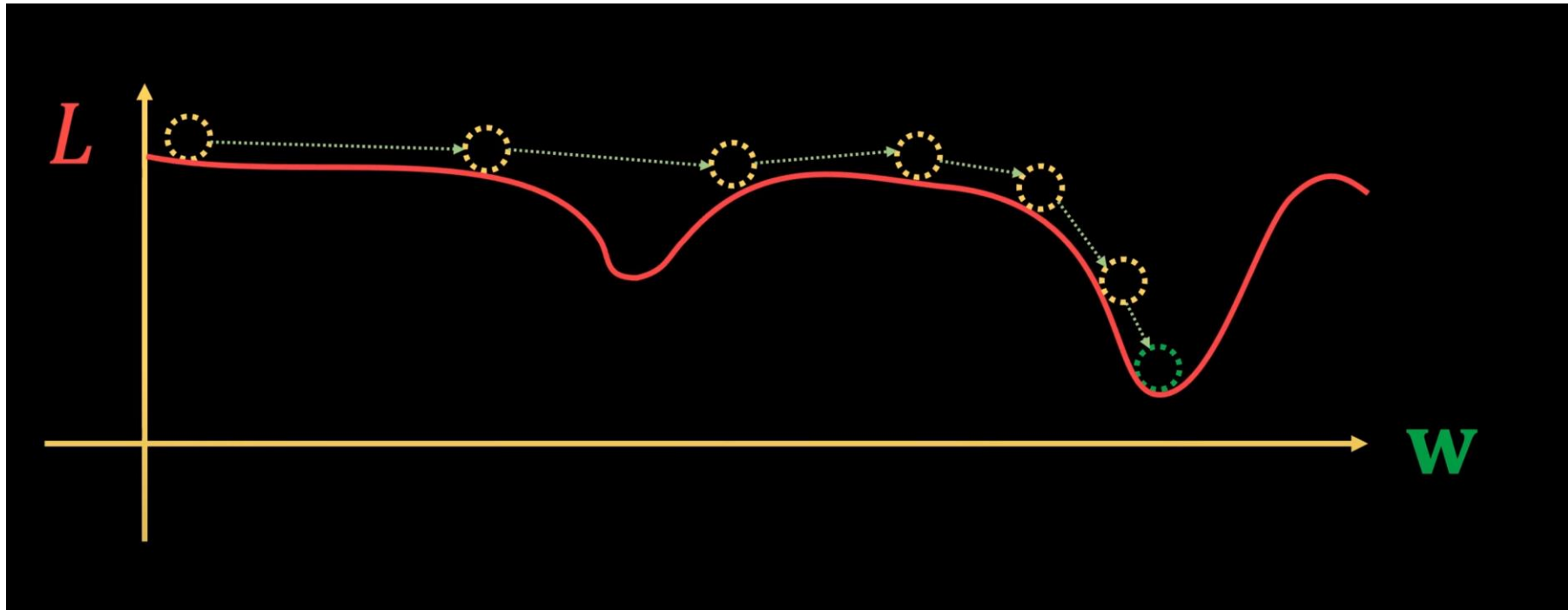
RMSPROP

$$\mathbf{v}_{t+1} = \beta \mathbf{v}_t + (1 - \beta) \mathbf{g}(\mathbf{w}_t)^2$$
$$\mathbf{w}_{t+1} = \mathbf{w}_t - \frac{\eta}{\epsilon + \sqrt{\mathbf{v}_{t+1}}} \mathbf{g}(\mathbf{w}_t)$$

Update rule



RMSPROP



A D A M

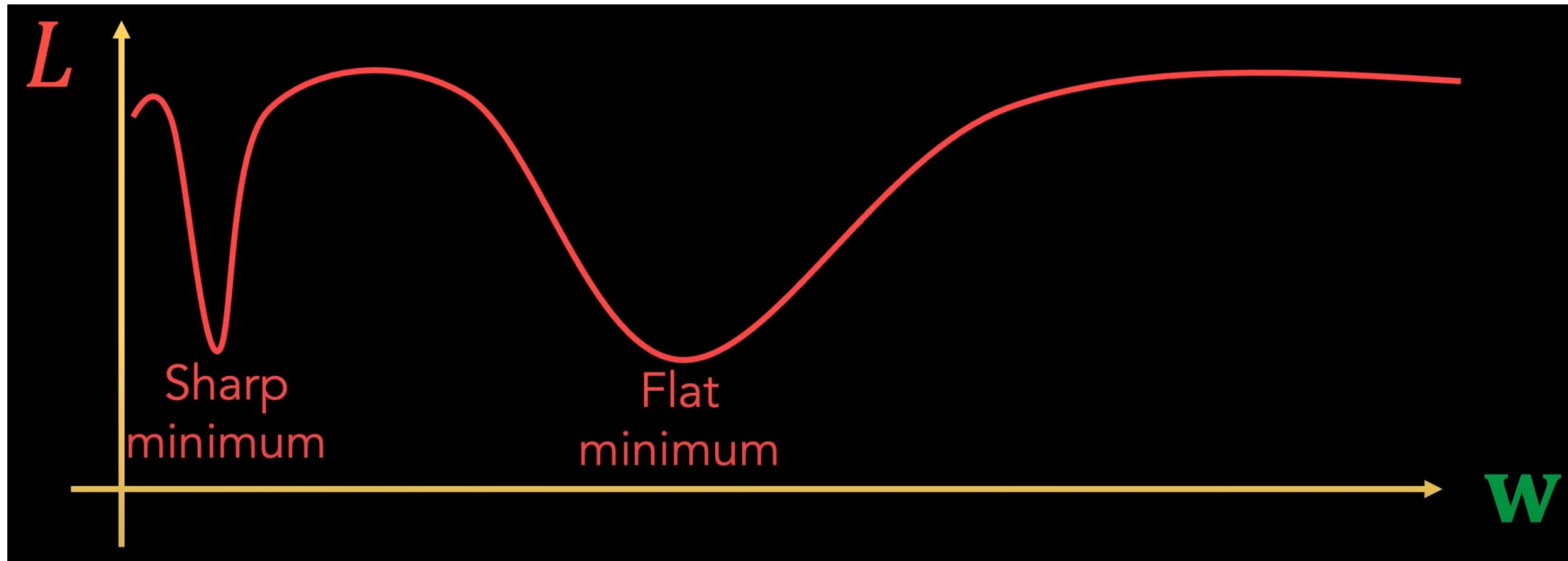
$$\mathbf{m}_{t+1} = \beta_1 \mathbf{m}_t + (1 - \beta_1) \mathbf{g}(\mathbf{w}_t) \quad \left| \quad \hat{\mathbf{m}}_{t+1} = \mathbf{m}_{t+1} / (1 - \beta_1^{t+1})\right.$$

$$\mathbf{v}_{t+1} = \beta_2 \mathbf{v}_t + (1 - \beta_2) \mathbf{g}(\mathbf{w}_t)^2 \quad \left| \quad \hat{\mathbf{v}}_{t+1} = \mathbf{v}_{t+1} / (1 - \beta_2^{t+1})\right.$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \frac{\eta}{\epsilon + \sqrt{\hat{\mathbf{v}}_{t+1}}} \hat{\mathbf{m}}_{t+1}$$



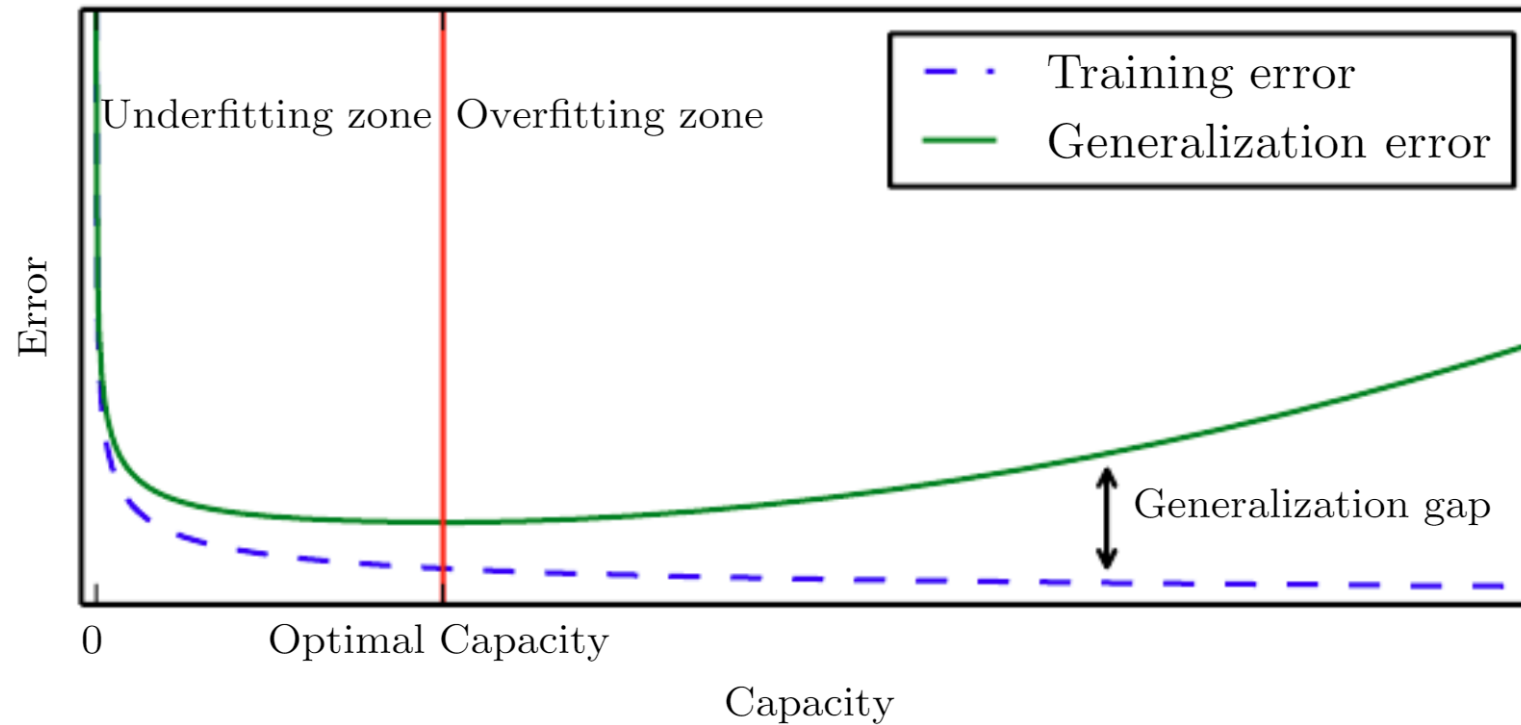
SGD VS ADAM



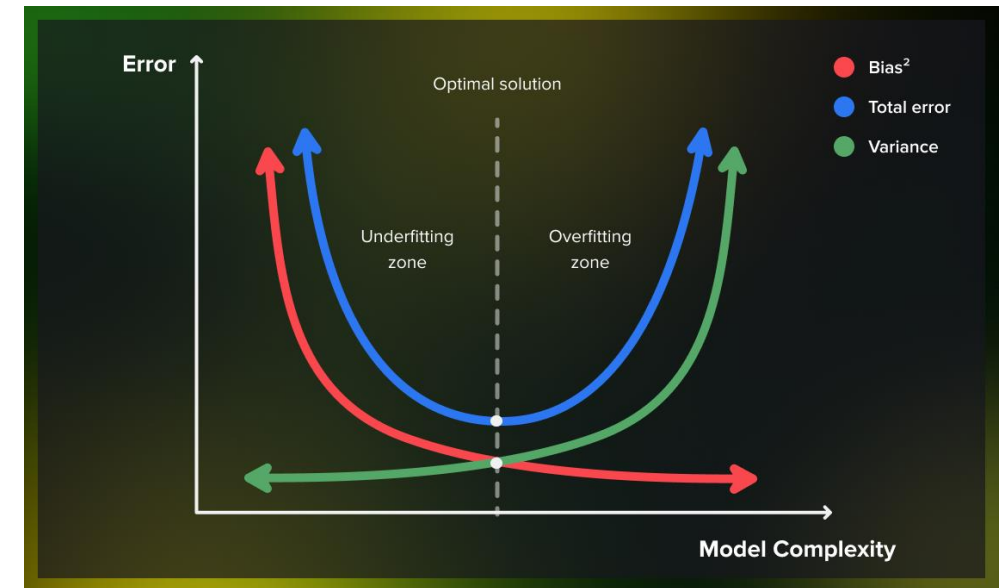
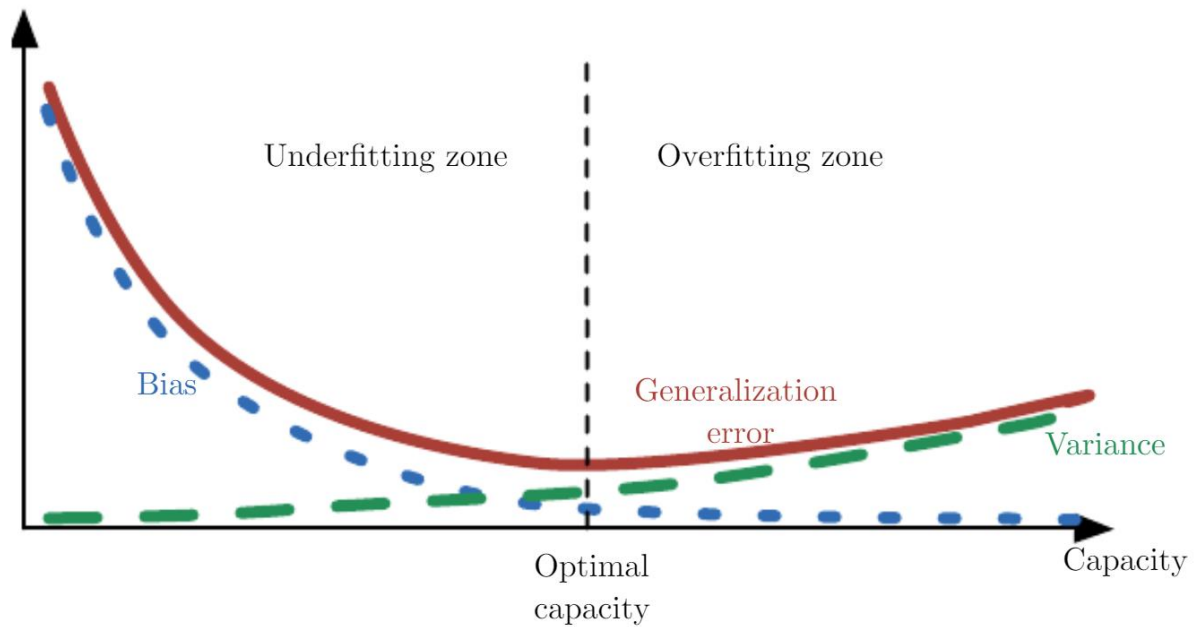
- 1: **given** $\alpha = 0.001, \beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}, \lambda \in \mathbb{R}$
- 2: **initialize** time step $t \leftarrow 0$, parameter vector $\theta_{t=0} \in \mathbb{R}^n$, first moment vector $m_{t=0} \leftarrow \mathbf{0}$, second moment vector $v_{t=0} \leftarrow \mathbf{0}$, schedule multiplier $\eta_{t=0} \in \mathbb{R}$
- 3: **repeat**
- 4: $t \leftarrow t + 1$
- 5: $\nabla f_t(\theta_{t-1}) \leftarrow \text{SelectBatch}(\theta_{t-1})$ ▷ select batch and return the corresponding gradient
- 6: $g_t \leftarrow \nabla f_t(\theta_{t-1}) + \lambda \theta_{t-1}$
- 7: $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$ ▷ here and below all operations are element-wise
- 8: $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$
- 9: $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ ▷ β_1 is taken to the power of t
- 10: $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ ▷ β_2 is taken to the power of t
- 11: $\eta_t \leftarrow \text{SetScheduleMultiplier}(t)$ ▷ can be fixed, decay, or also be used for warm restarts
- 12: $\theta_t \leftarrow \theta_{t-1} - \eta_t \left(\alpha \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) + \lambda \theta_{t-1} \right)$
- 13: **until** stopping criterion is met
- 14: **return** optimized parameters θ_t

2 - OVERFITTING AND UNDERFITTING

TRAINING VS GENERALIZATION ERROR

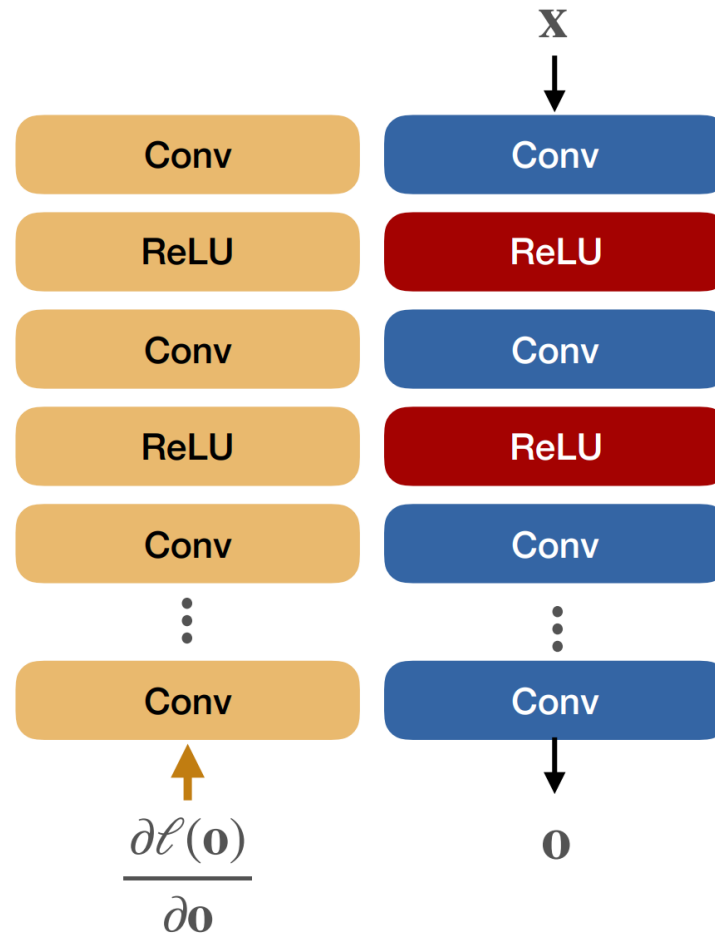


BIAS VS VARIANCE



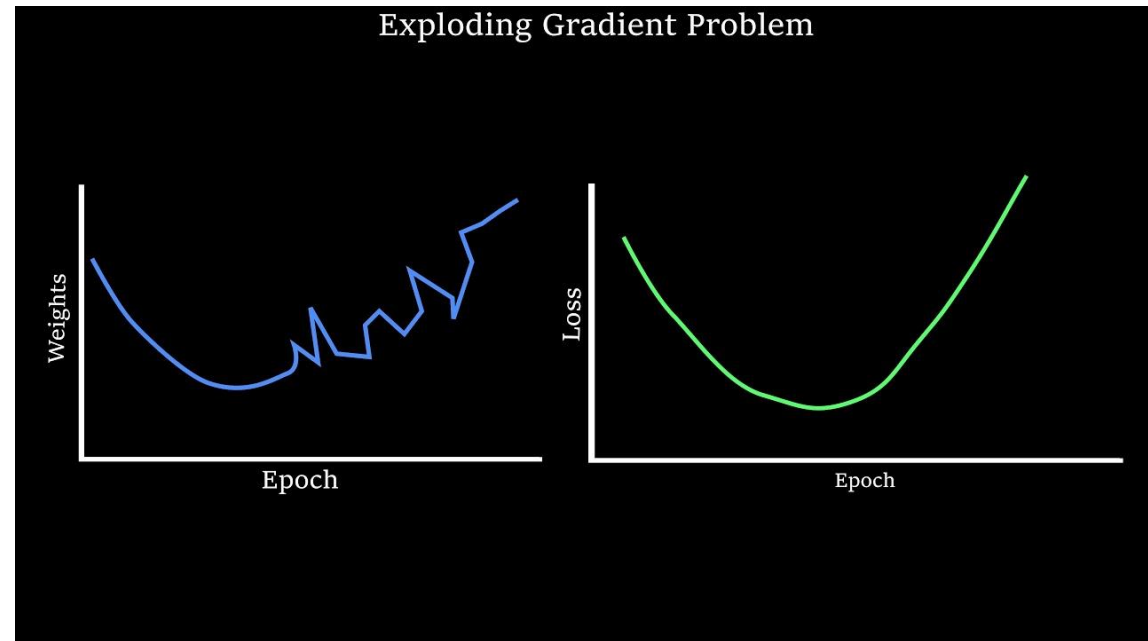
3 - NORMALIZATION

HOW TO PREVENT VANISHING OR EXPLODING GRADIENTS?



DETECT EXPLODING GRADIENTS

- Plot loss
- Plot weight and gradient magnitude per layer

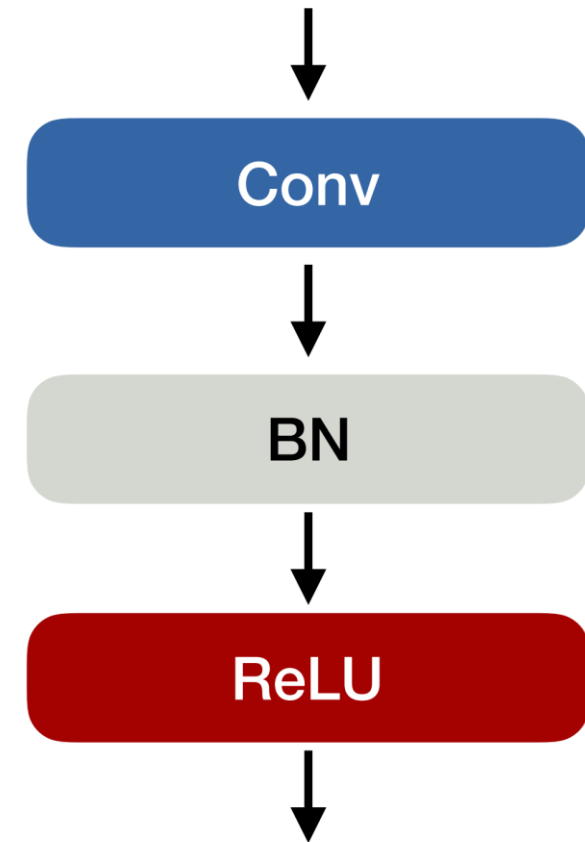
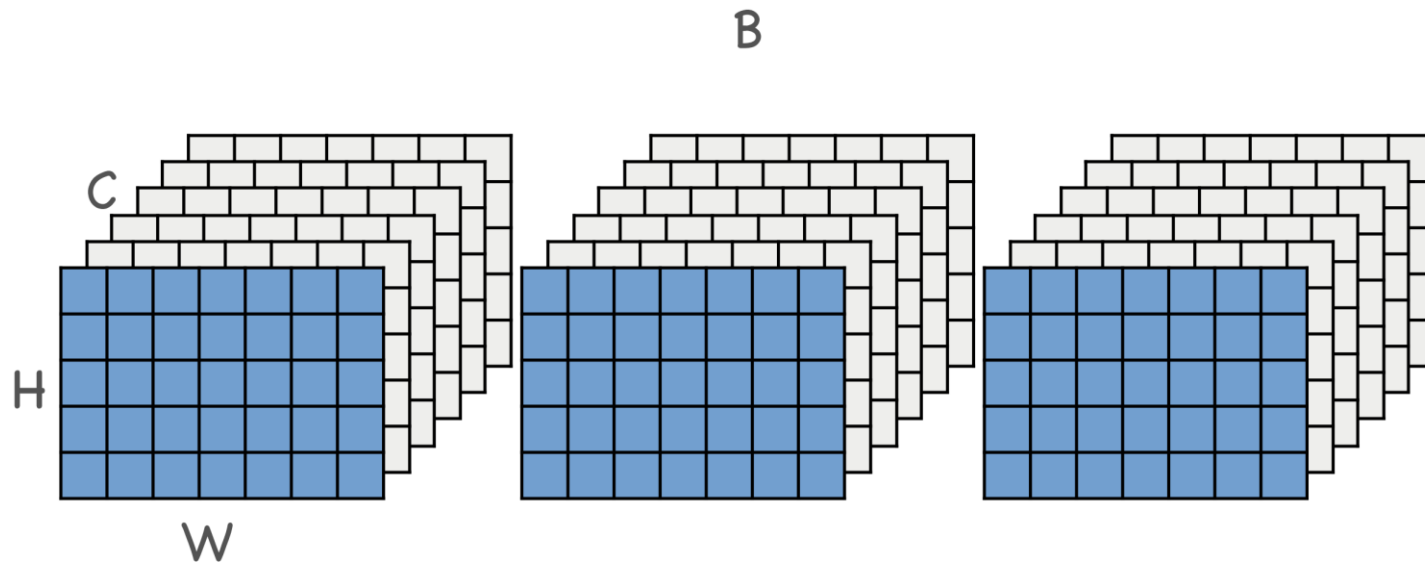


DETECT VANISHING GRADIENTS

- Plot loss
- Plot weight and gradient magnitude per layer



BATCH NORM



BATCH NORM

Normalize by channel-wise mean μ_c and standard deviation σ_c

$$\mathbf{Z} \in \mathbb{R}^{B \times W \times H \times C}$$



$$\frac{\mathbf{Z}_{k,x,y,c} - \mu_c}{\sigma_c}$$

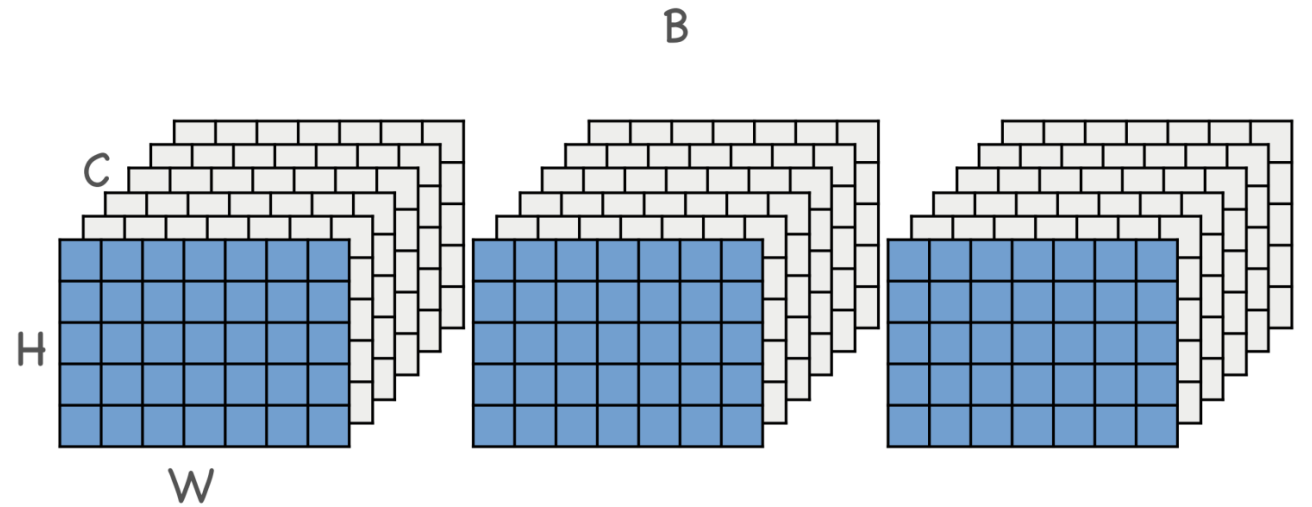
$$\mu_c = \frac{1}{BWH} \sum_{k,x,y} \mathbf{Z}_{k,x,y,c}$$

$$\sigma_c^2 = \frac{1}{BWH} \sum_{k,x,y} (\mathbf{Z}_{k,x,y,c} - \mu_c)^2$$



BATCH NORM AND BATCH SIZE

- Large batch size works better
- More stable mean and standard deviation



BATCH NORM AT TEST TIME

- Compute mean and standard deviation on training set using running average



```
# train batch norm  
model.train()  
  
# eval batch norm  
model.eval()
```

$$\mathbf{Z} \in \mathbb{R}^{1 \times W \times H \times C}$$

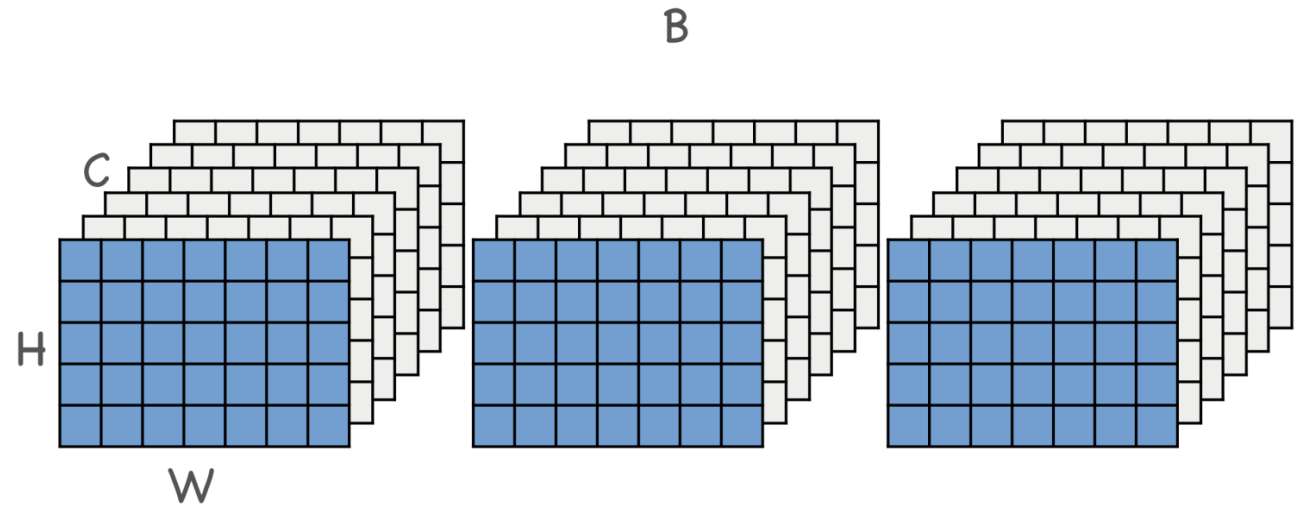


$$\frac{\mathbf{Z}_{k,x,y,c} - \mu_c}{\sigma_c}$$

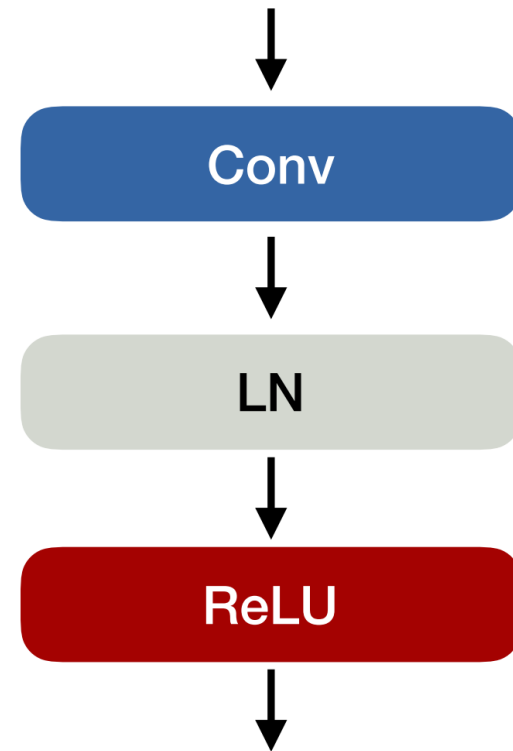
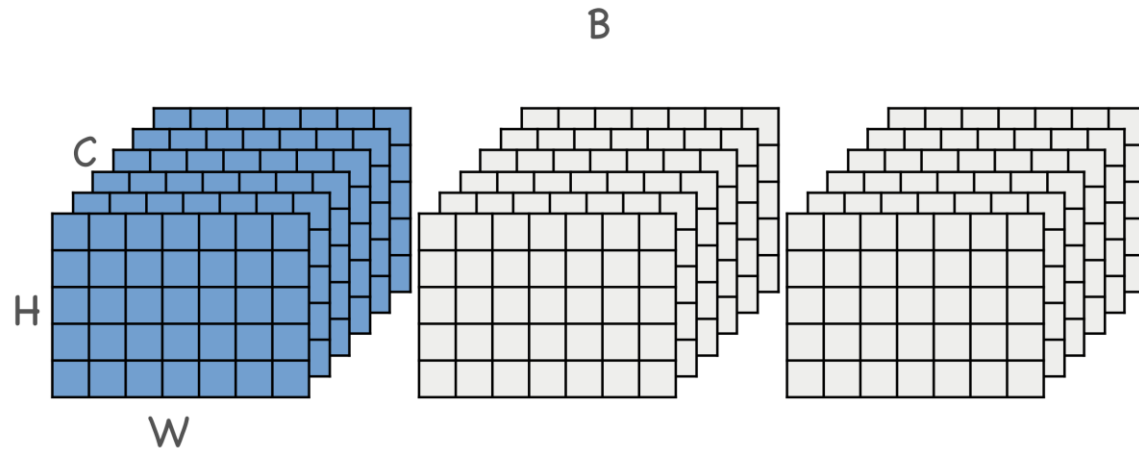


BENEFITS FROM BATCH NORM

- The good:
 - Regularizes the network
 - Handles badly scaled weights
- The bad:
 - Mixes the gradient information between samples



LAYER NORM



LAYER NORM

Normalize by image-wise mean μ_k and standard deviation σ_k

$$\mathbf{Z} \in \mathbb{R}^{B \times W \times H \times C}$$



$$\frac{\mathbf{Z}_{k,x,y,c} - \mu_k}{\sigma_k}$$

$$\mu_k = \frac{1}{WHC} \sum_{x,y,c} \mathbf{Z}_{k,x,y,c}$$

$$\sigma_k^2 = \frac{1}{WHC} \sum_{x,y,c} (\mathbf{Z}_{k,x,y,c} - \mu_k)^2$$



LAYER NORM

- Make activations zero mean and unit variance without collecting statistics across batches

$$\mathbf{Z} \in \mathbb{R}^{B \times W \times H \times C}$$



$$\frac{\mathbf{Z}_{k,x,y,c} - \mu_k}{\sigma_k}$$

$$\mu_k = \frac{1}{WHC} \sum_{x,y,c} \mathbf{Z}_{k,x,y,c}$$

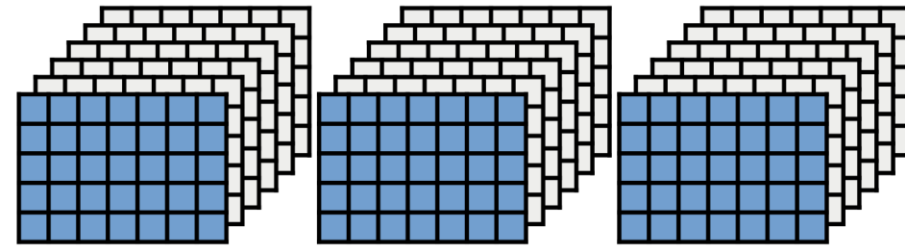
$$\sigma_k^2 = \frac{1}{WHC} \sum_{x,y,c} (\mathbf{Z}_{k,x,y,c} - \mu_k)^2$$



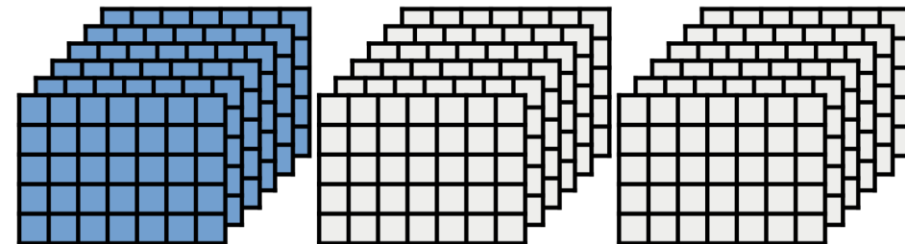
LAYER VS BATCH NORM

- No summary statistics:
 - Training and testing are the same
- Works well for sequence models
- Does not scale activations individually

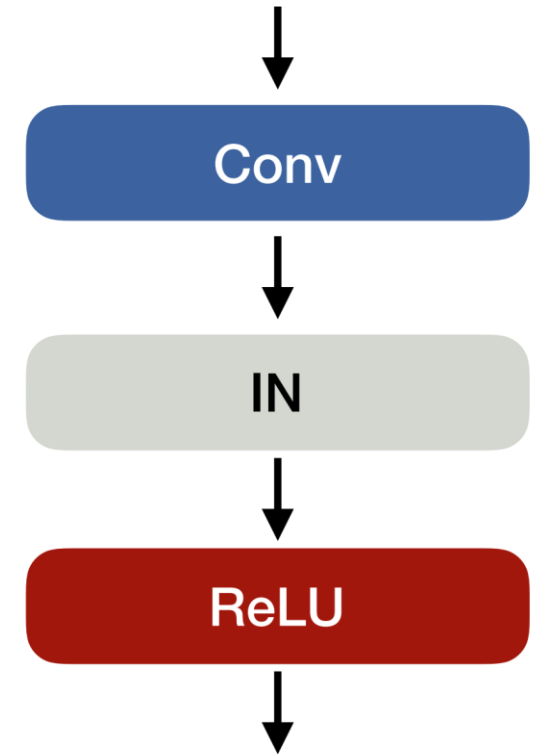
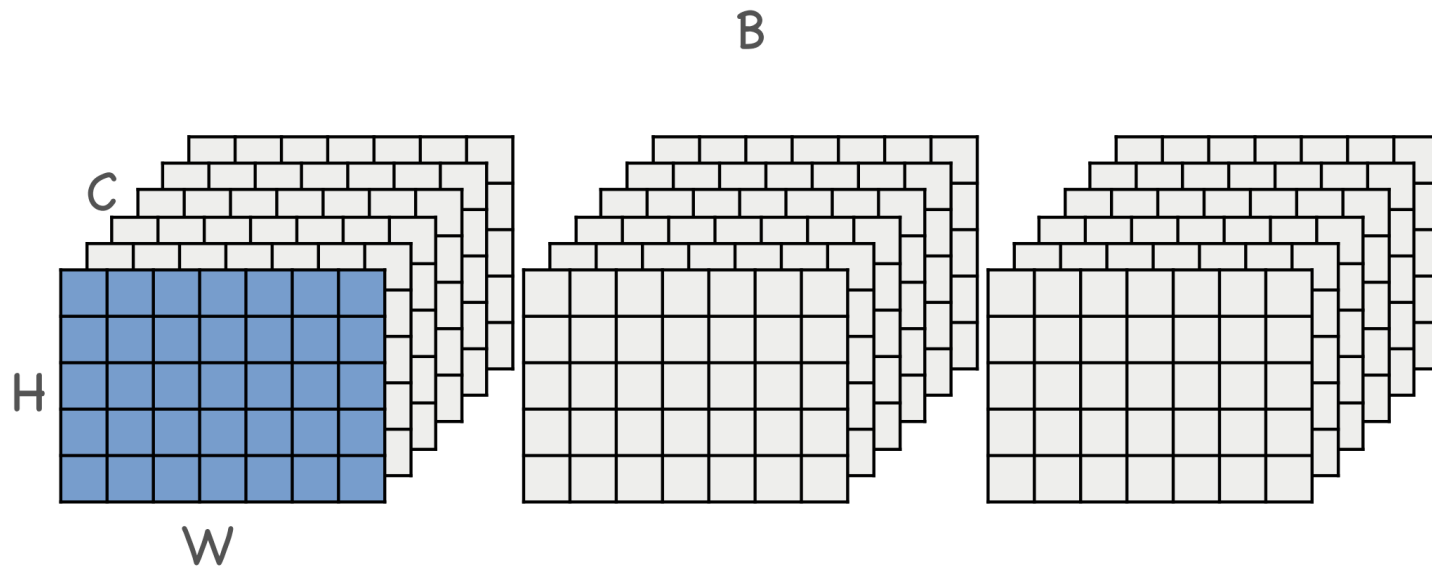
batch norm



layer norm



INSTANCE NORM



INSTANCE NORM

Normalize by spatial
mean μ_{kc} and standard
deviation σ_{kc}

$$\mathbf{Z} \in \mathbb{R}^{B \times W \times H \times C}$$



$$\frac{\mathbf{Z}_{k,x,y,c} - \mu_{kc}}{\sigma_{kc}}$$

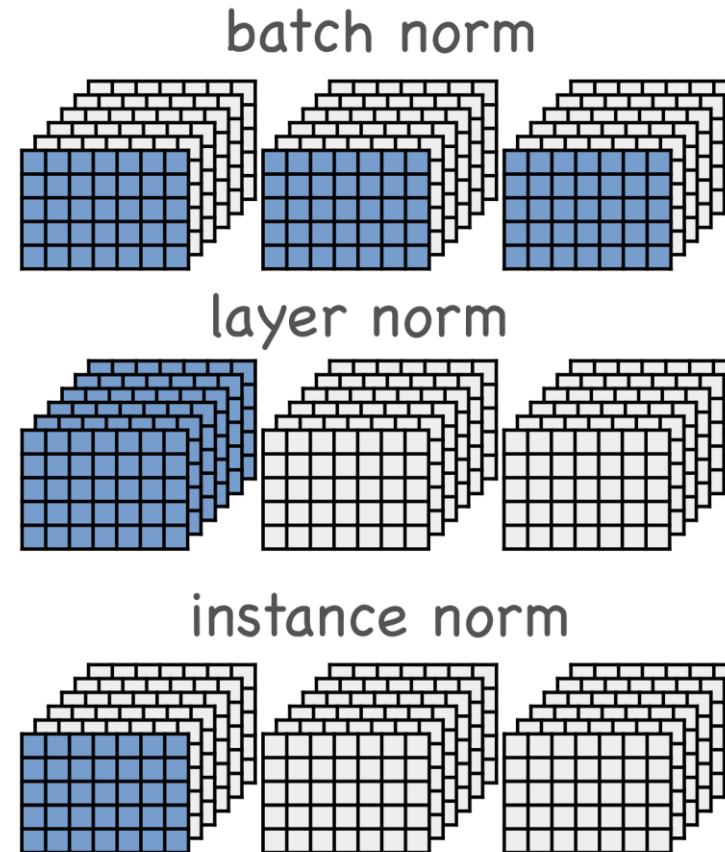
$$\mu_{kc} = \frac{1}{WH} \sum_{x,y} \mathbf{Z}_{k,x,y,c}$$

$$\sigma_{kc}^2 = \frac{1}{WH} \sum_{x,y} (\mathbf{Z}_{k,x,y,c} - \mu_{kc})^2$$



BATCH VS LAYER VS INSTANCE NORM

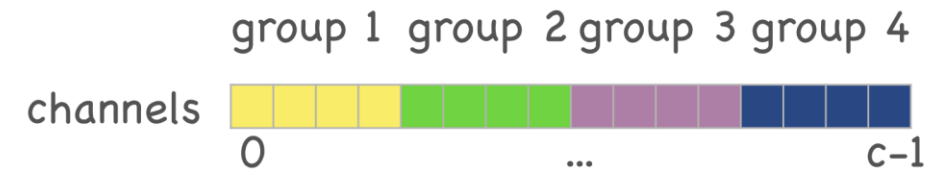
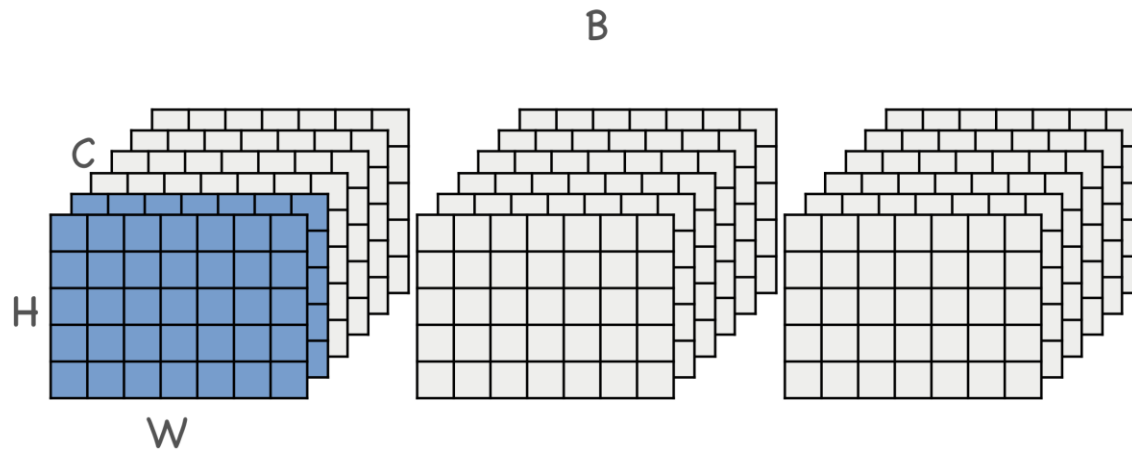
- No summing over batches
- Works well for graphics applications
- Not used much in recognition
- Unstable statistics



APPLICATION OF INSTANCE NORM - STYLIZATION



GROUP NORM



$$\mathbf{Z} \in \mathbb{R}^{B \times W \times H \times C}$$

$$\frac{\mathbf{Z}_{k,x,y,c} - \mu_{kg}}{\sigma_{kg}} \quad g = \lfloor c/G \rfloor$$

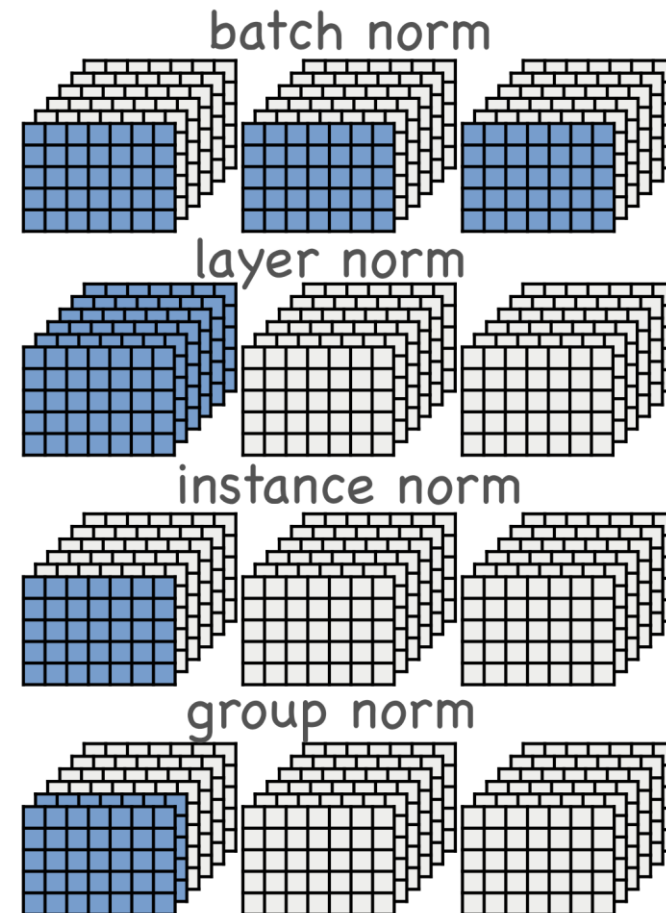
$$\mu_{kg} = \frac{1}{WHG} \sum_{c=gG}^{(g+1)G-1} \sum_{x,y} \mathbf{Z}_{k,x,y,c}$$

$$\sigma_{kg}^2 = \frac{1}{WHG} \sum_{c=gG}^{(g+1)G-1} \sum_{x,y} (\mathbf{Z}_{k,x,y,c} - \mu_{kg})^2$$

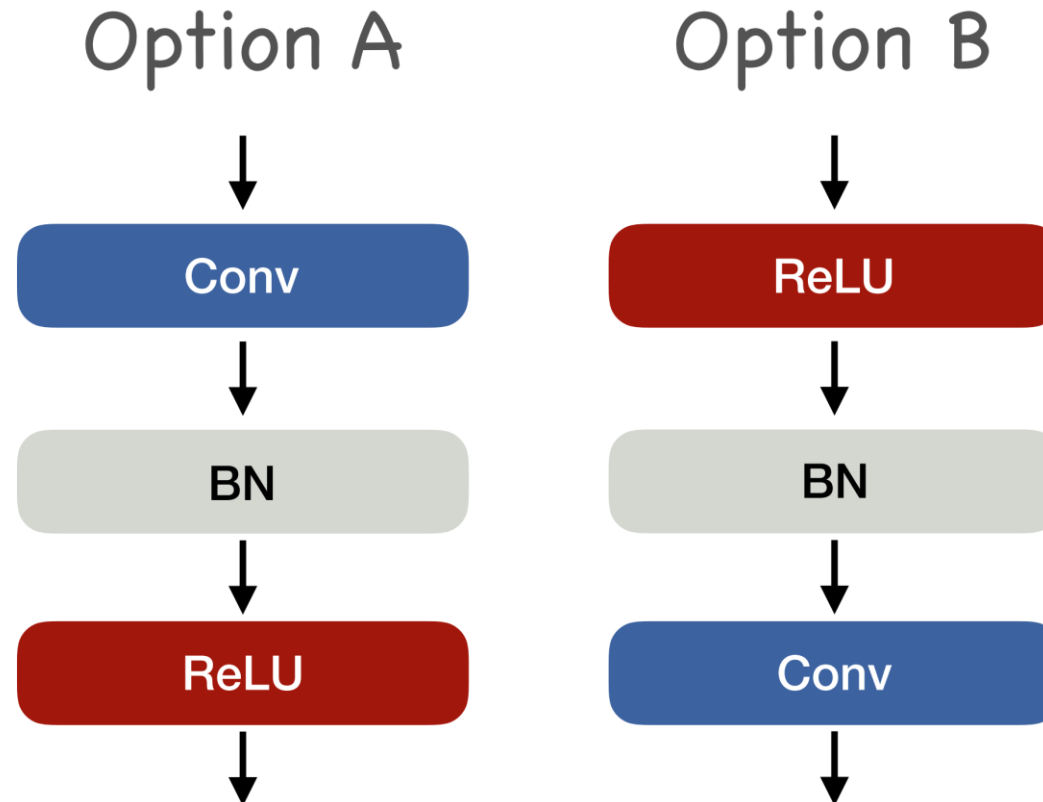


COMPARISON WITH OTHER NORMS

- More stable statistics than instance norm ($G=C$)
- Not all channels tied as in layer norm ($G=1$)

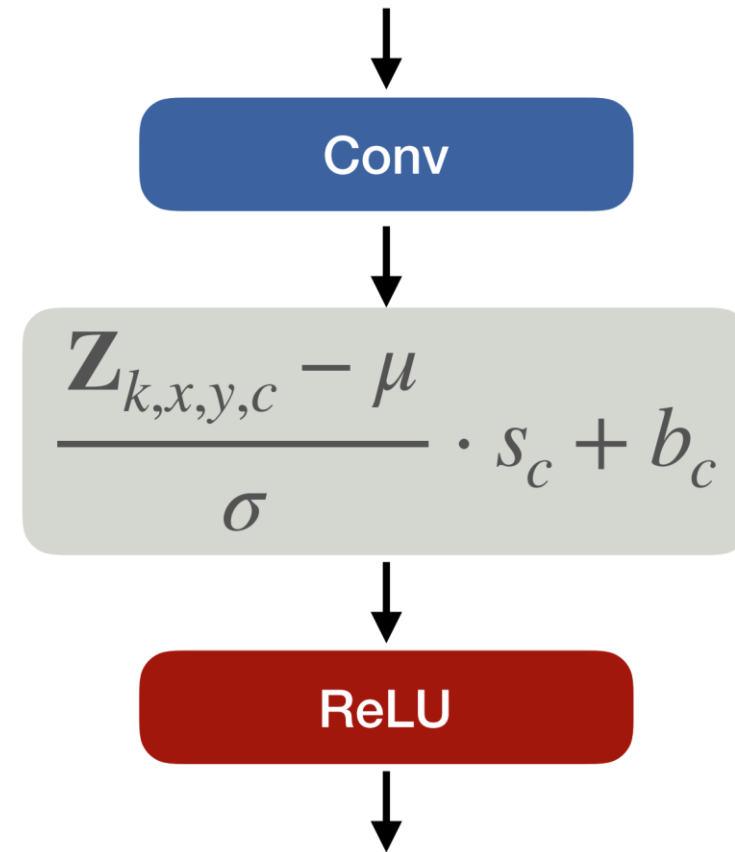


WHERE TO ADD NORMALIZATION?



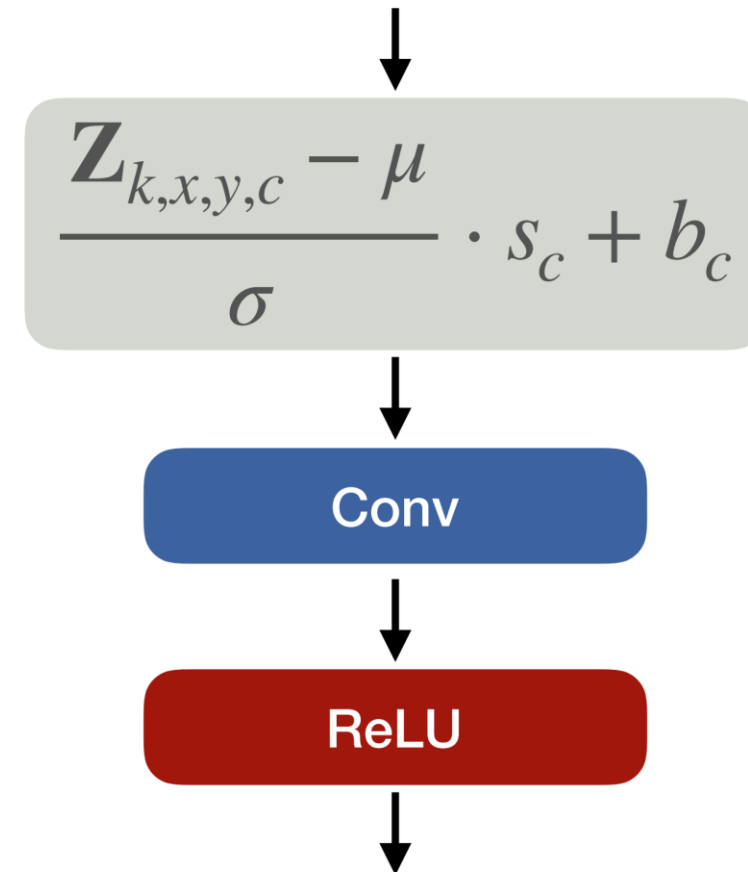
OPTION A

- No bias in conv
- Activations are zero mean
- Half of activations are zeroed in the ReLU
- Solution:
 - Learn a scale and bias after norm



OPTION B

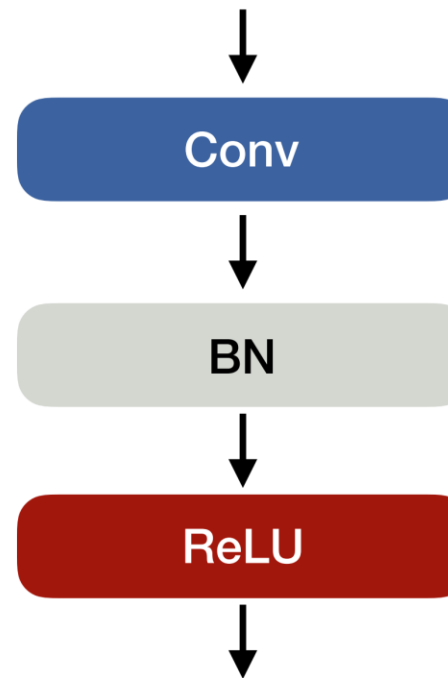
- Scale and bias after norm are optional



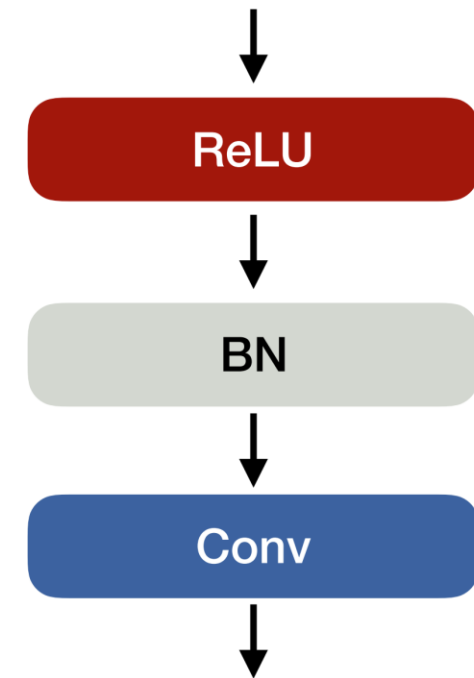
A VS B

- Both work
- A is more popular
- B is easier

Option A

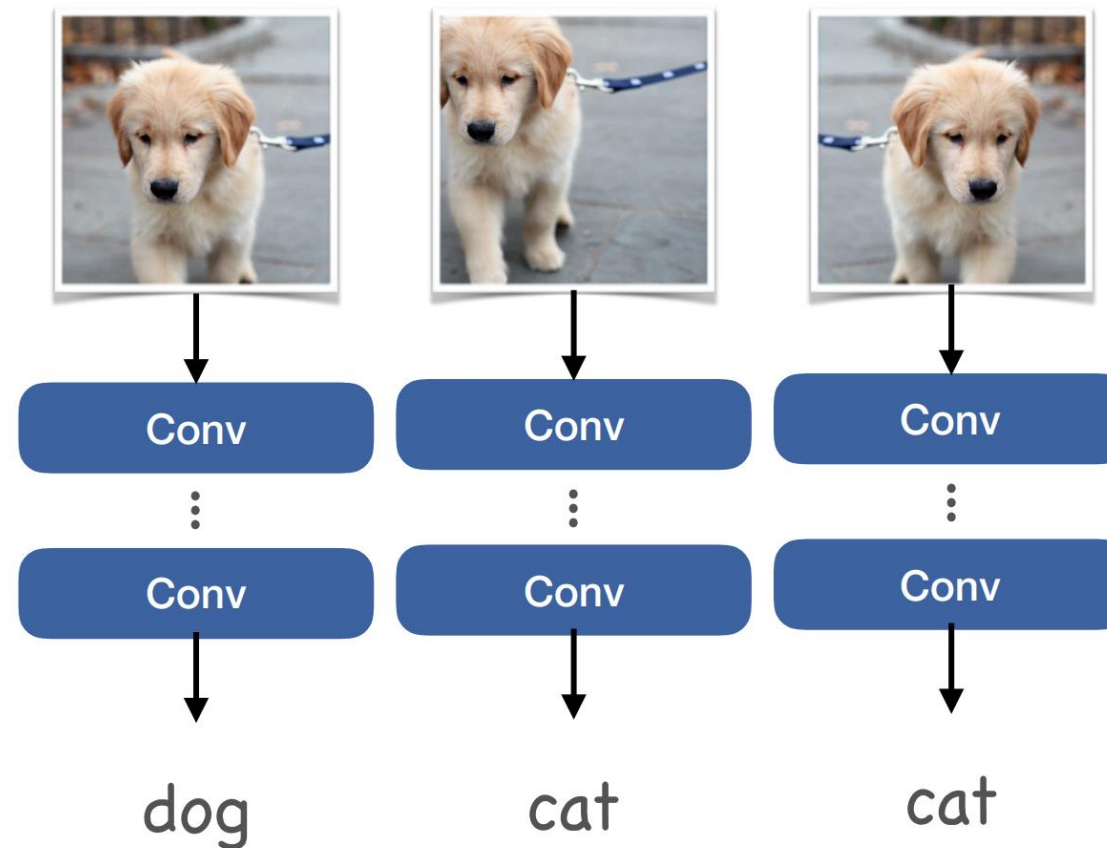


Option B



4 - DATA AUGMENTATION

SIGNS OF OVERFITTING



DATA AUGMENTATION

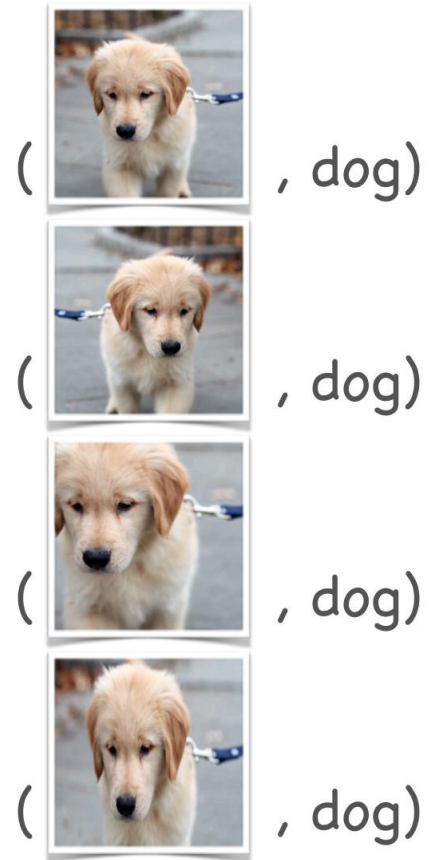
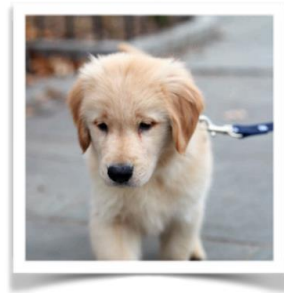
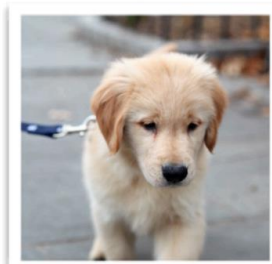


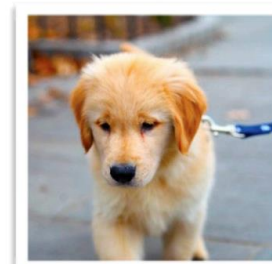
IMAGE AUGMENTATIONS



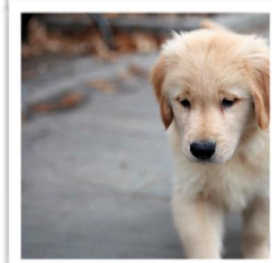
flip



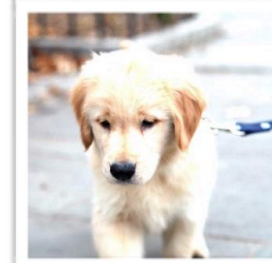
saturation



shift



brightness



scale



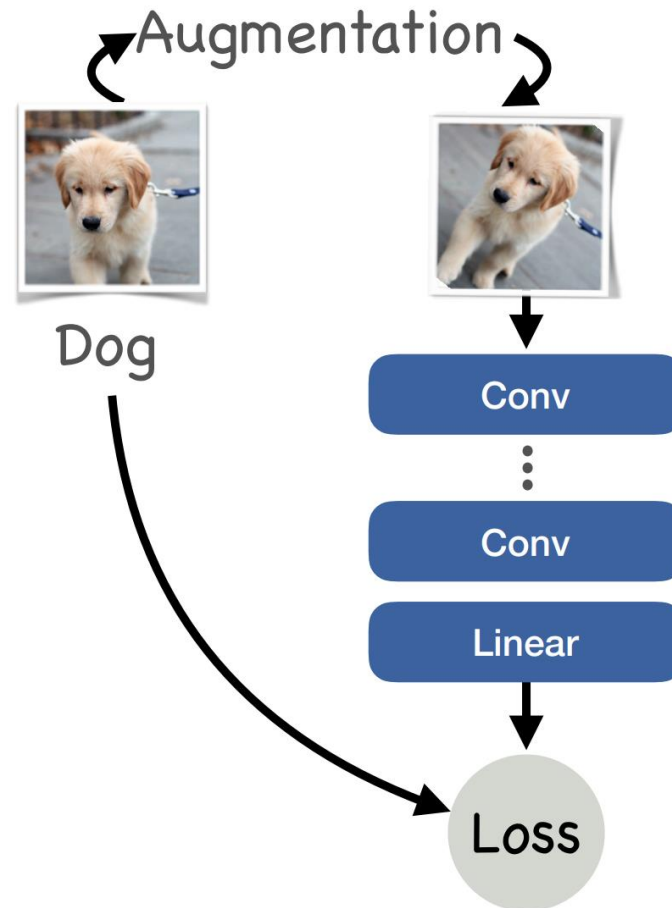
rotate



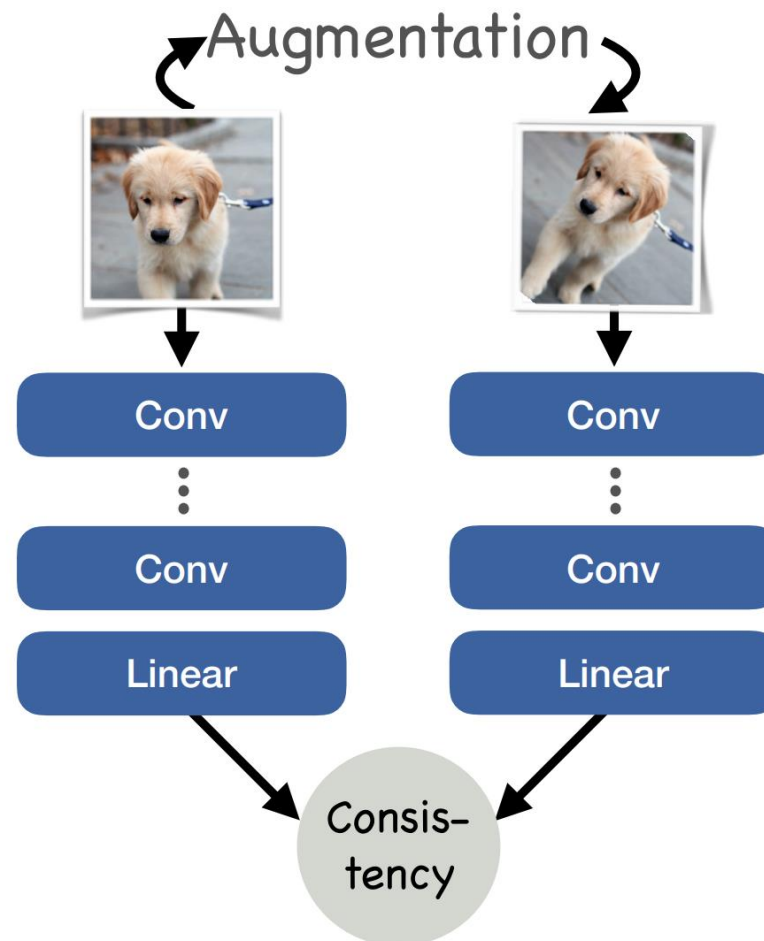
tint/hue



TRAINING WITH AUGMENTATIONS

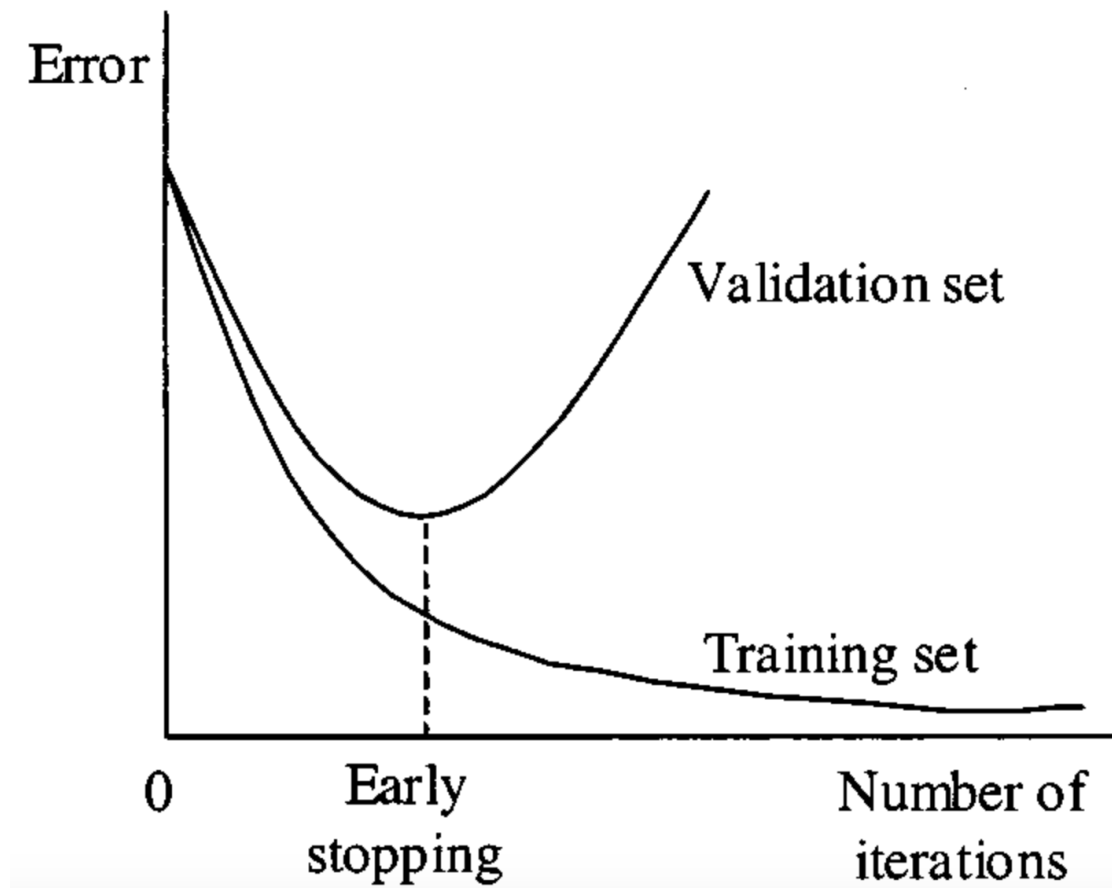


UNSUPERVISED DATA AUGMENTATION



5 - E A R L Y S T O P P I N G

WHEN TO STOP



PATIENCE

Algorithm 7.1 The early stopping meta-algorithm for determining the best amount of time to train. This meta-algorithm is a general strategy that works well with a variety of training algorithms and ways of quantifying error on the validation set.

Let n be the number of steps between evaluations.

Let p be the “patience,” the number of times to observe worsening validation set error before giving up.

Let θ_o be the initial parameters.

$\theta \leftarrow \theta_o$

$i \leftarrow 0$

$j \leftarrow 0$

$v \leftarrow \infty$

$\theta^* \leftarrow \theta$

$i^* \leftarrow i$

while $j < p$ **do**

 Update θ by running the training algorithm for n steps.

$i \leftarrow i + n$

$v' \leftarrow \text{ValidationSetError}(\theta)$

if $v' < v$ **then**

$j \leftarrow 0$

$\theta^* \leftarrow \theta$

$i^* \leftarrow i$

$v \leftarrow v'$

else

$j \leftarrow j + 1$

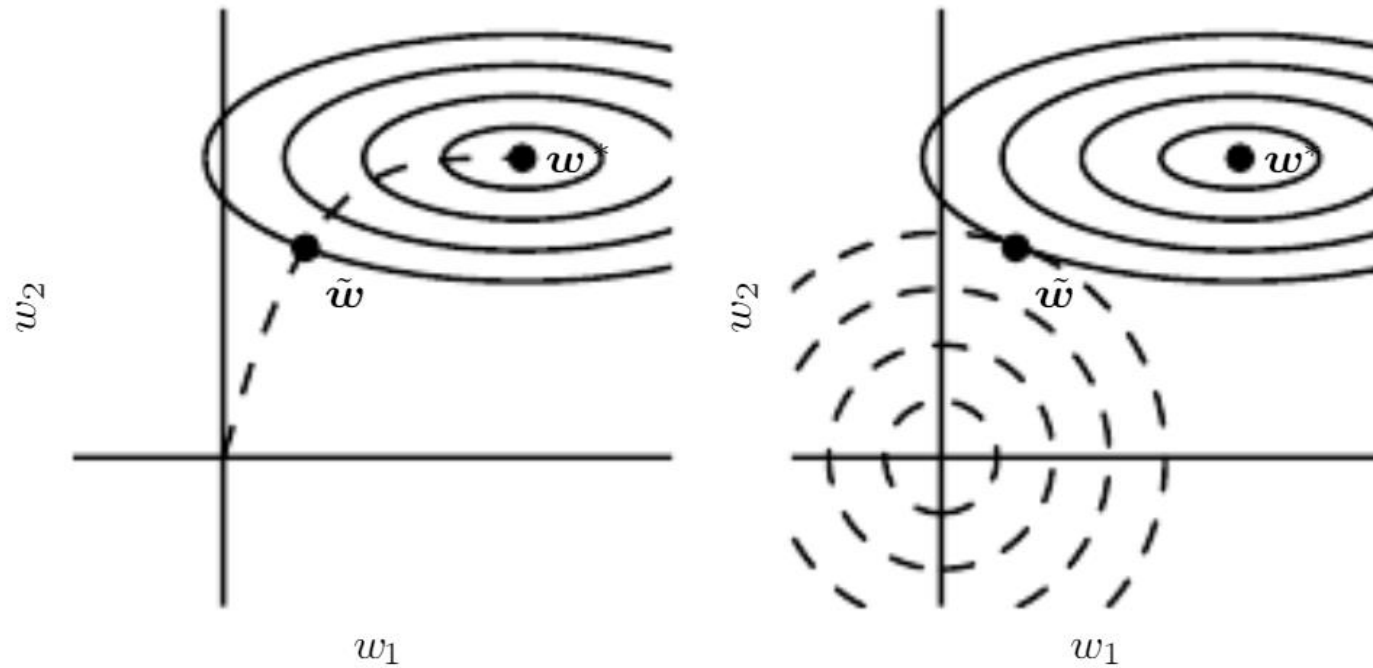
end if

end while

Best parameters are θ^* , best number of training steps is i^* .



EARLY STOPPING AS A REGULARIZER

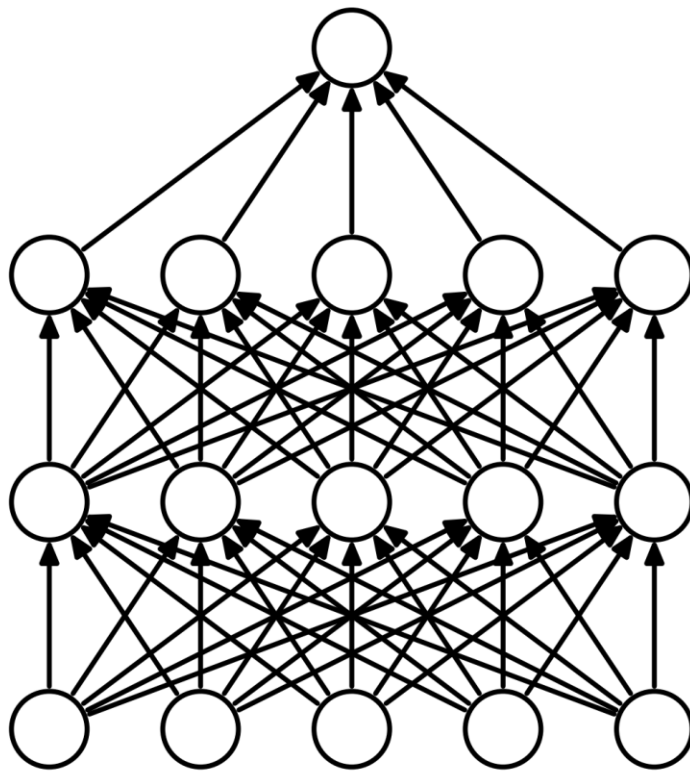


STOPPING TOO EARLY

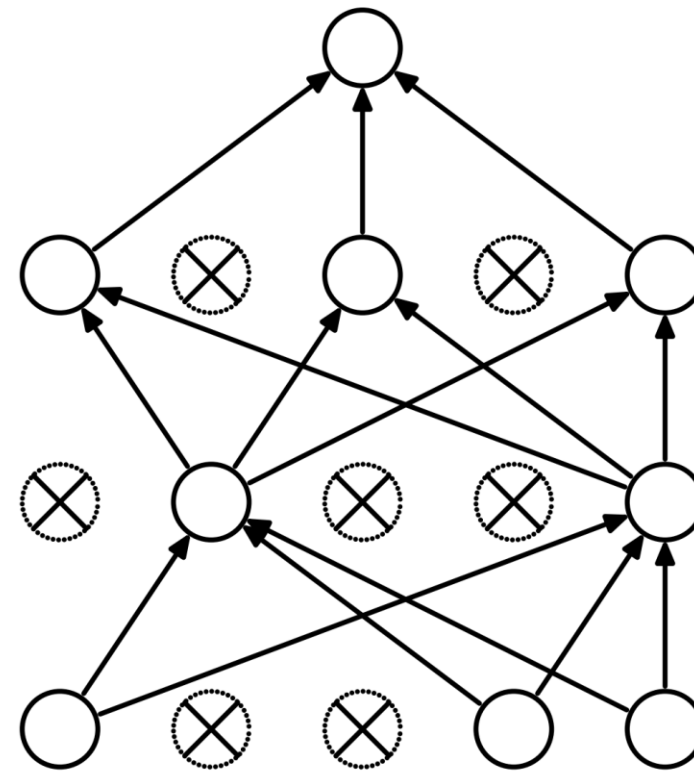


6 - D R O P O U T

DROPOUT



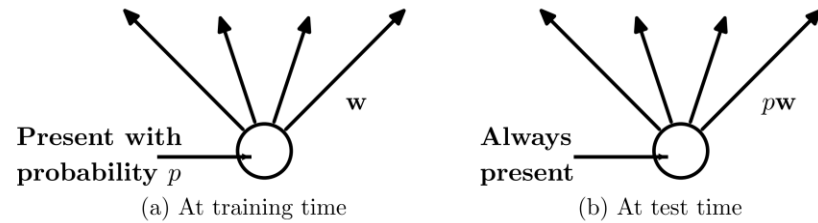
(a) Standard Neural Net



(b) After applying dropout.

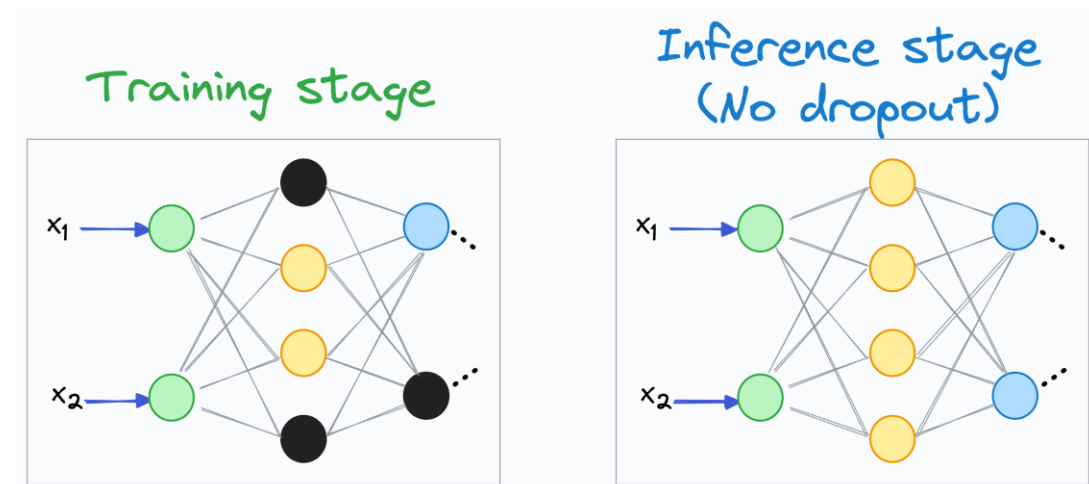


TRAIN VS EVALUATION



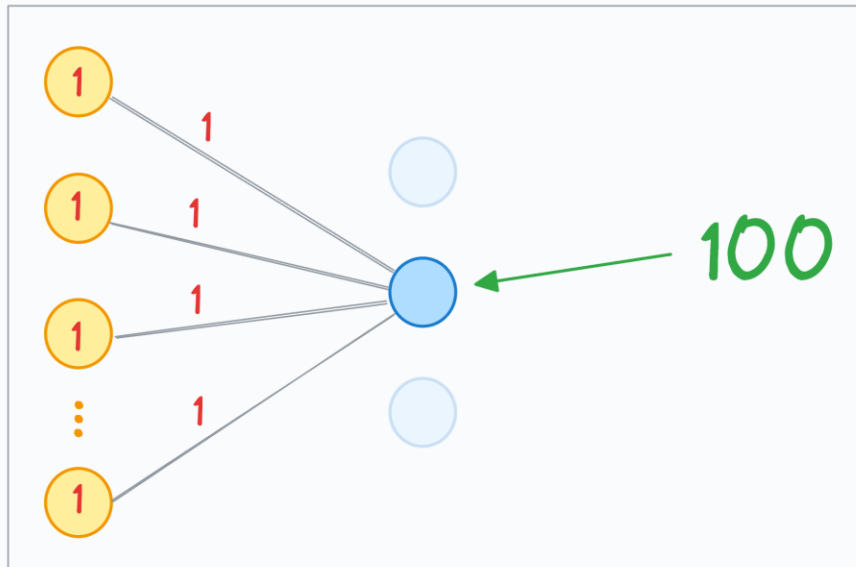
```
# train batch norm
model.train()

# eval batch norm
model.eval()
```

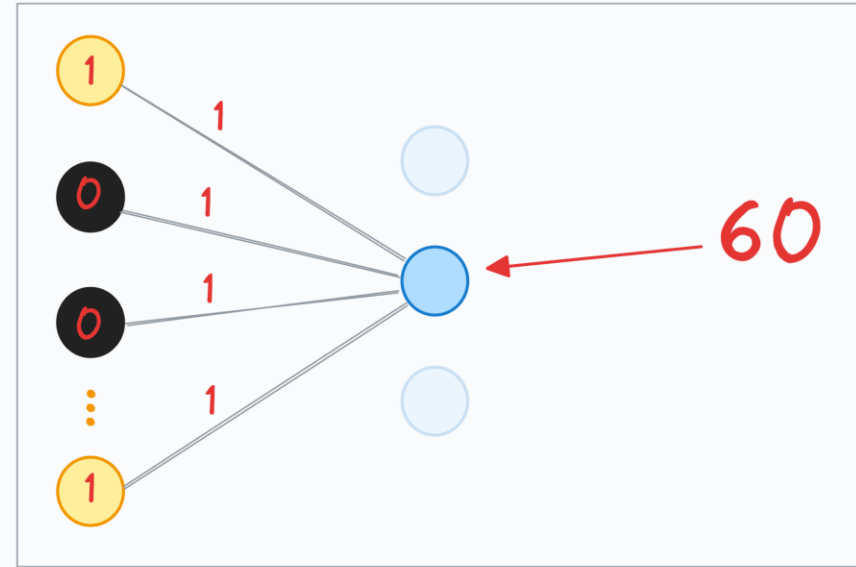


PROBLEM WITH SCALE OF ACTIVATIONS

Inference stage
(No dropout)

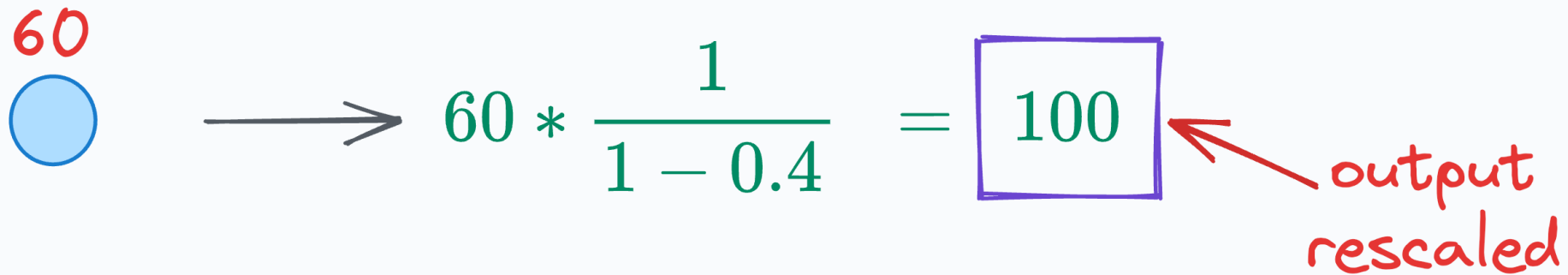


Training stage



SCALE FACTOR

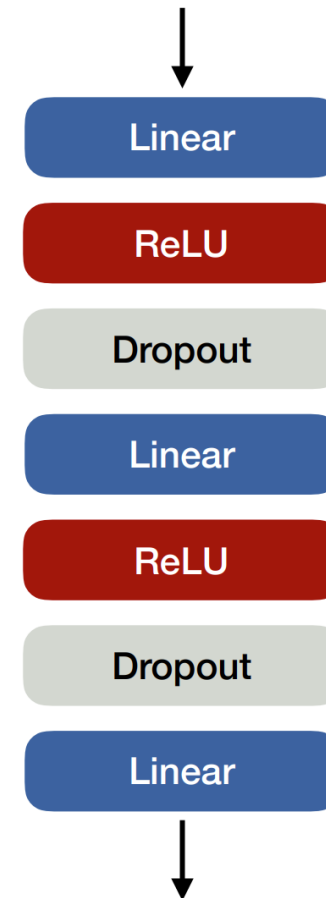
Scale activations by $\frac{1}{1 - \alpha}$


$$60 \rightarrow 60 * \frac{1}{1 - 0.4} = 100$$

output rescaled

WHERE TO ADD DROPOUT?

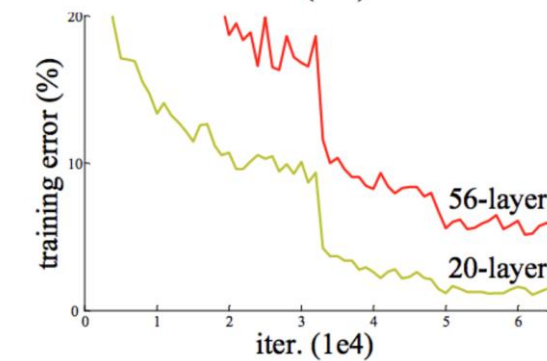
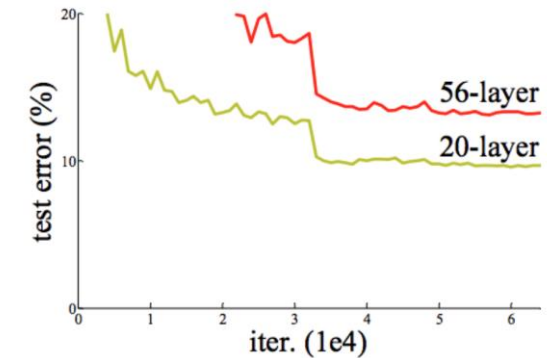
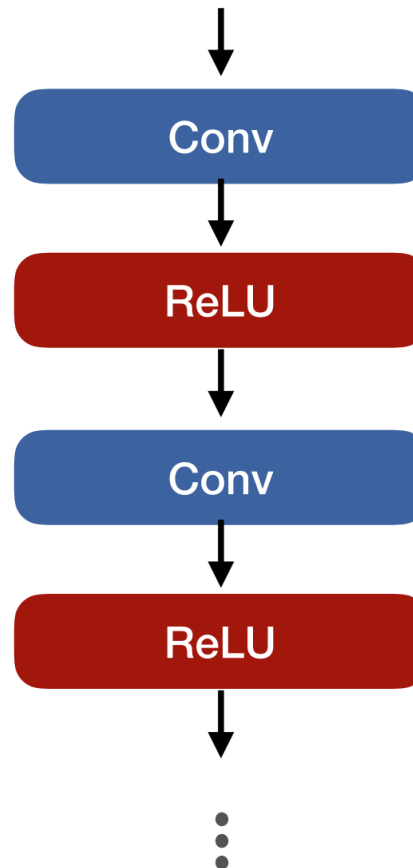
- Before any large fully connected layer
- Before some 1x1 convolutions
- Not before general convolutions



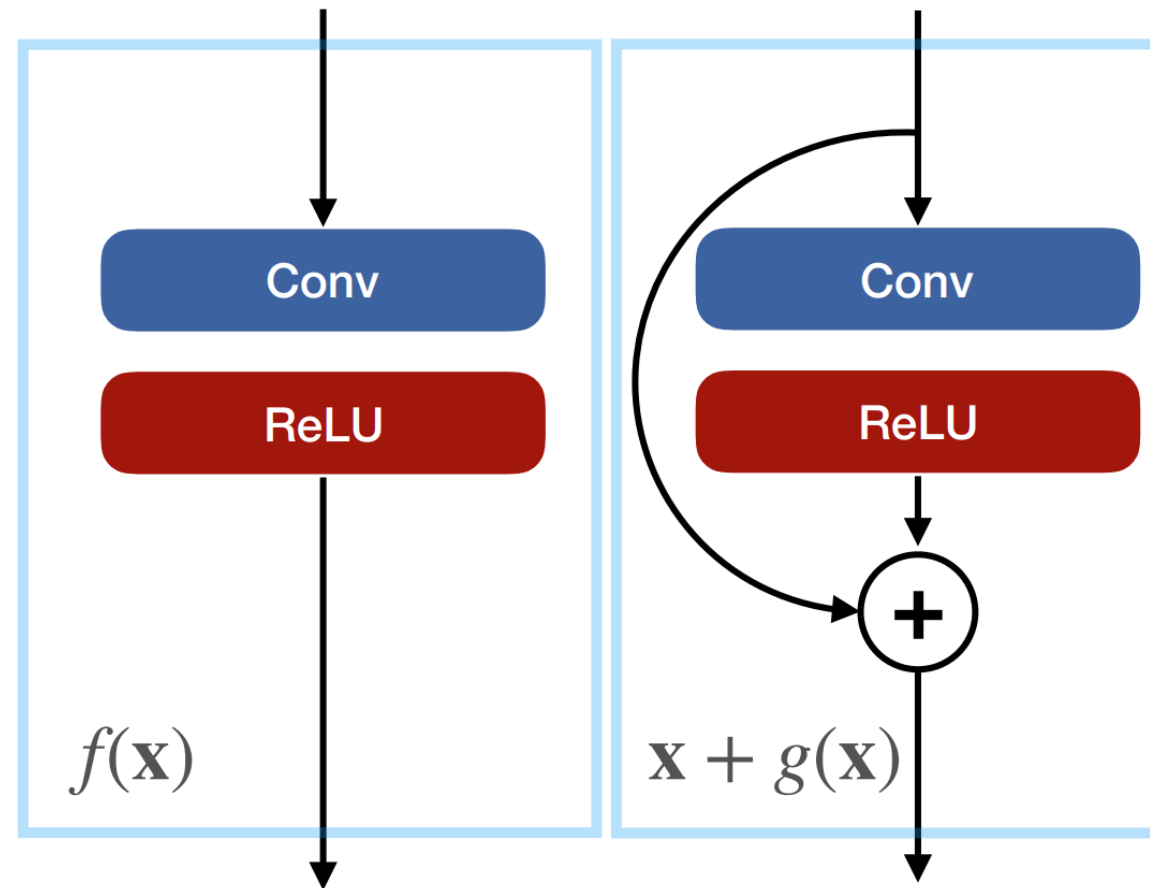
7 - RESIDUAL CONNECTIONS

WHAT HAPPENS TO DEEPER NETWORKS?

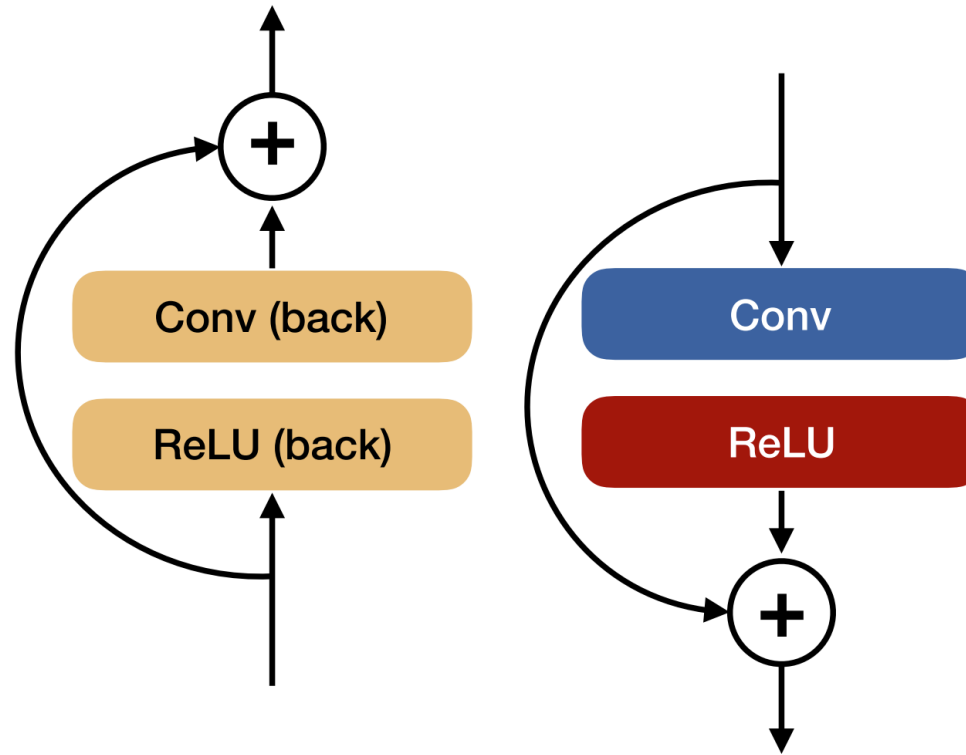
- Without normalization:
max-depth 10-12 layers
- With normalization: max-
depth 20-30



RESIDUAL CONNECTION - FORWARD

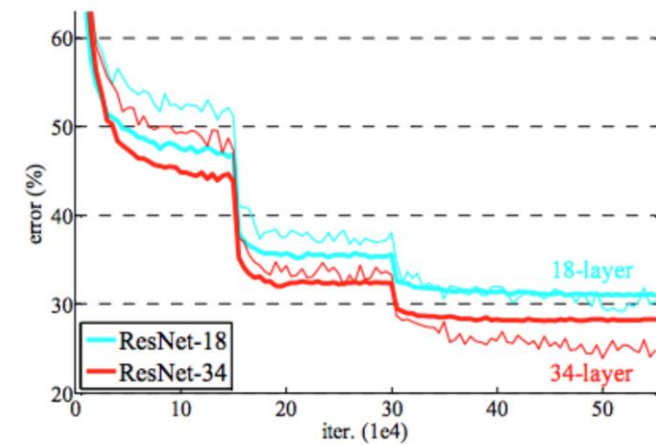
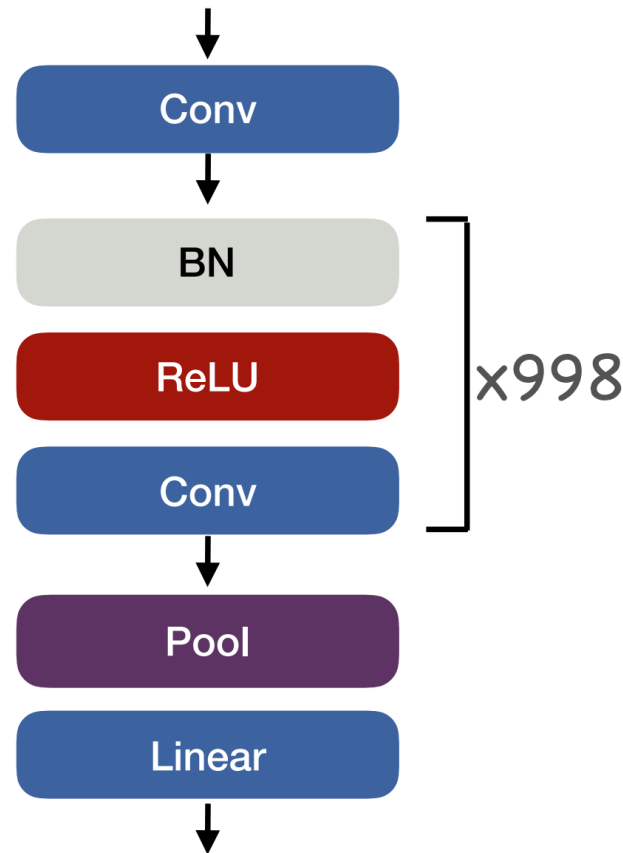


RESIDUAL CONNECTIONS - BACKWARD

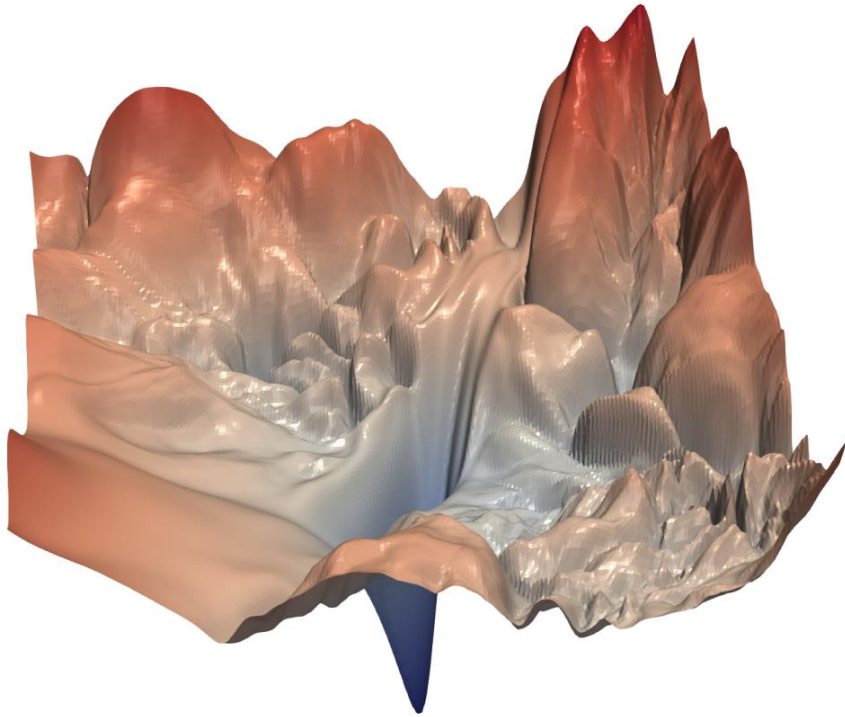


HOW WELL RESIDUAL CONNECTIONS WORK?

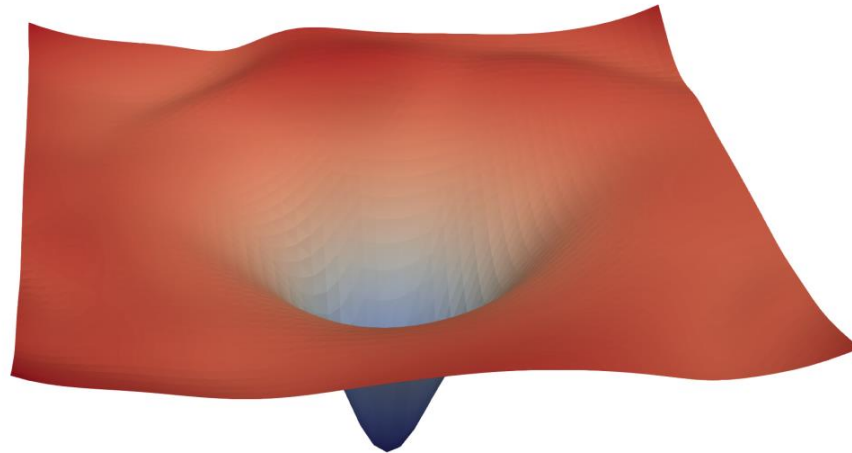
- New depth: 1000 layers



LOSS LANDSCAPE WITH RESIDUAL CONNECTIONS



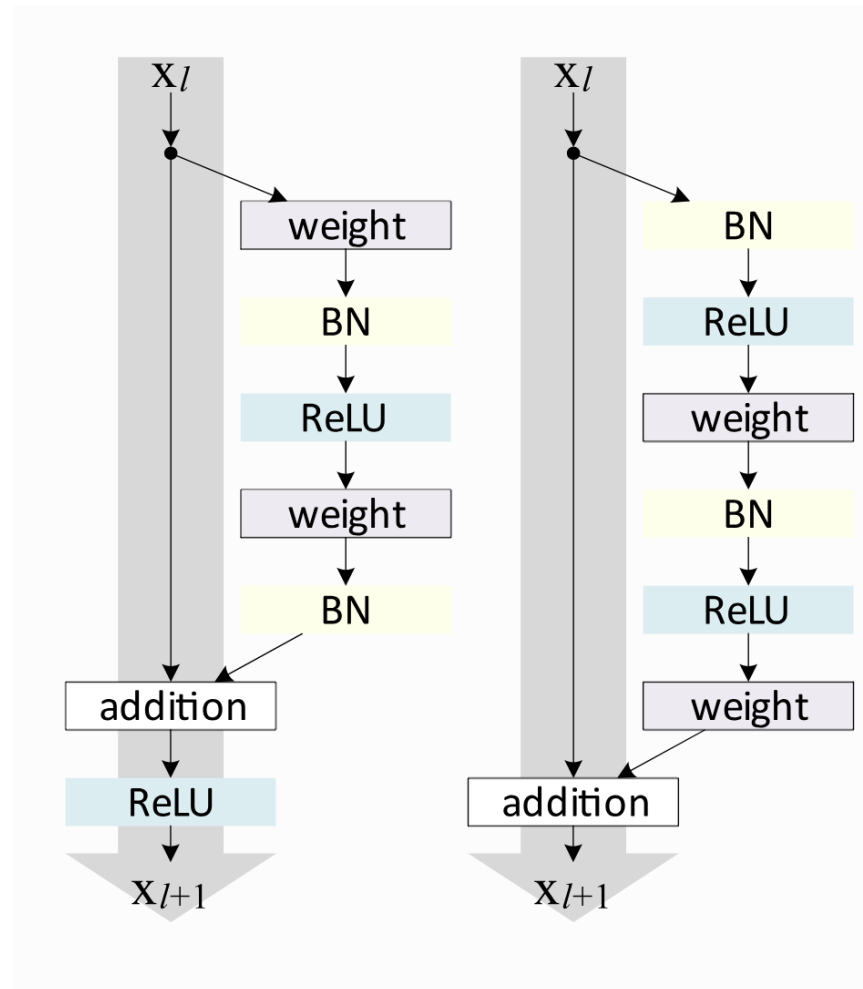
(a) without skip connections



(b) with skip connections

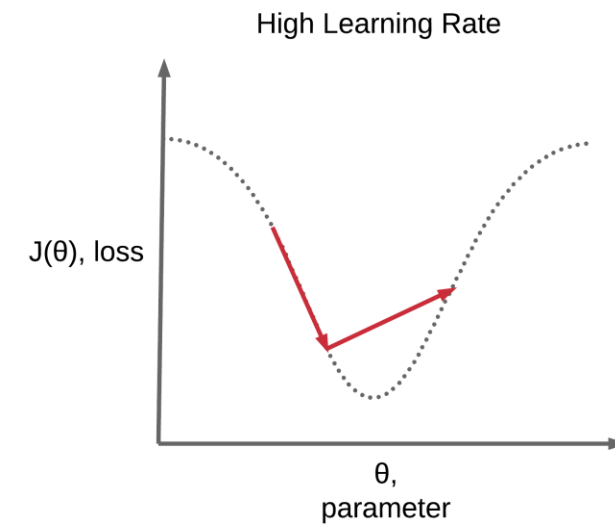
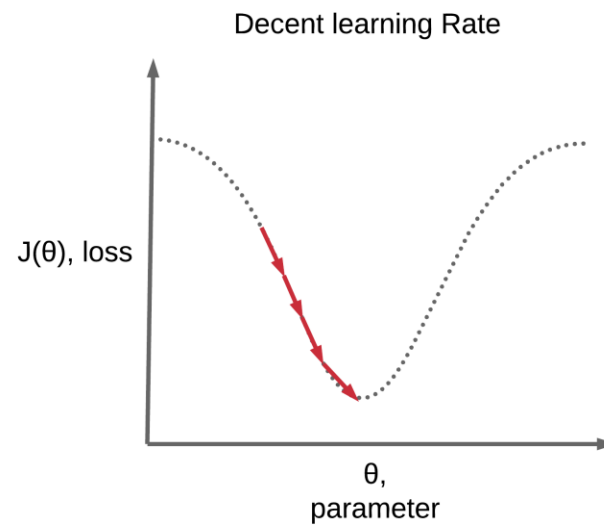
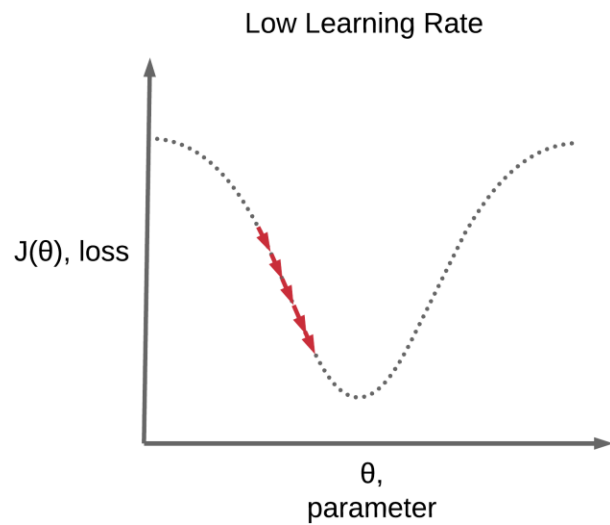


WHERE TO APPLY RESIDUAL CONNECTIONS?

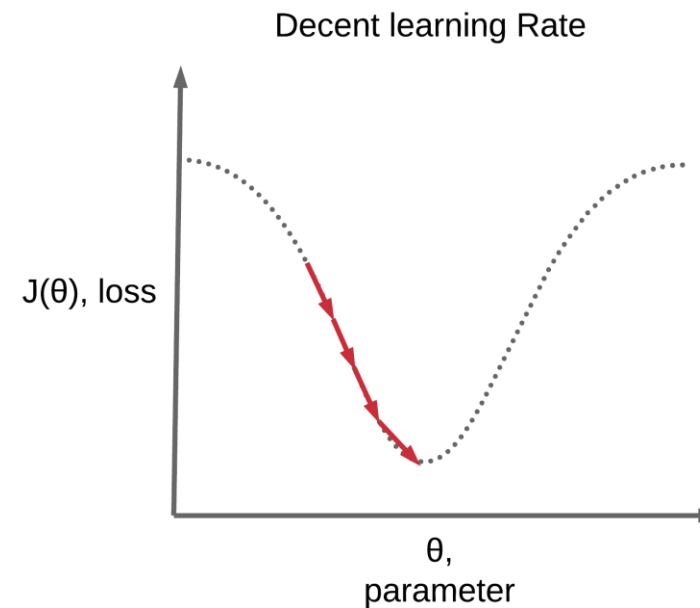
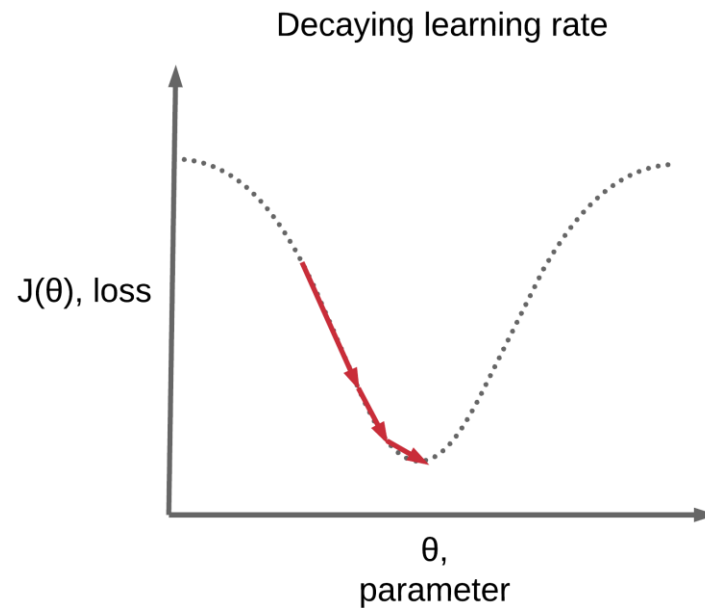


8 - LEARNING RATE SCHEDULERS

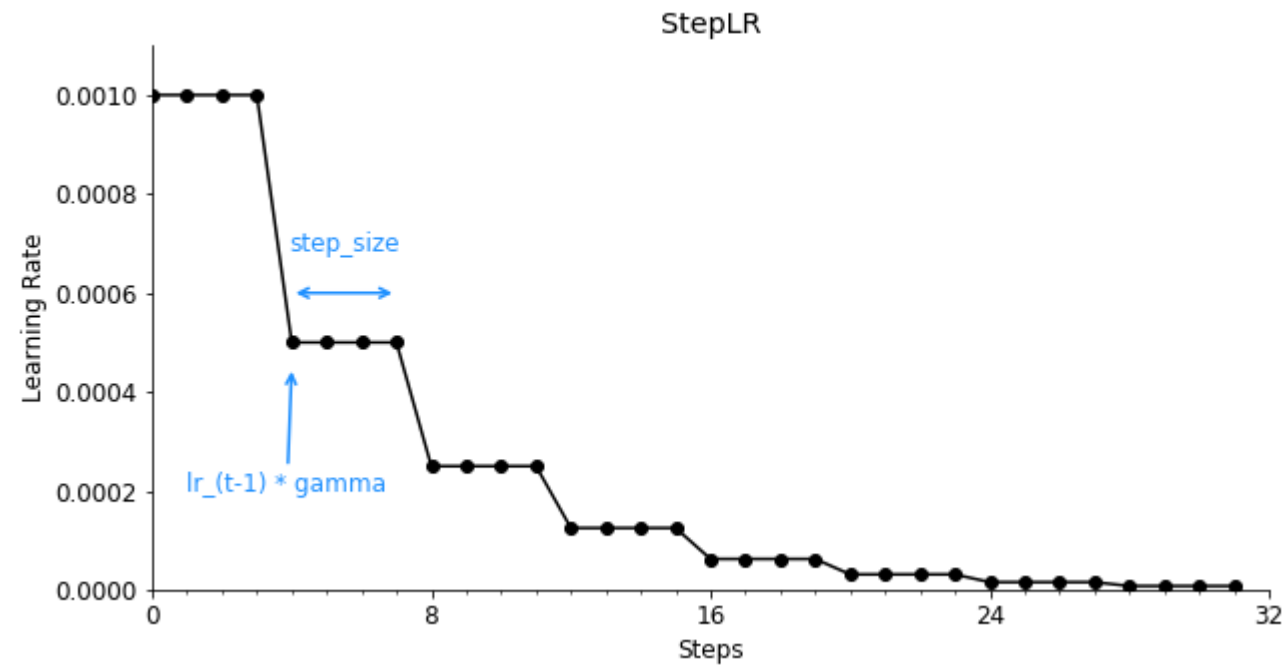
OPTIMAL LEARNING RATE



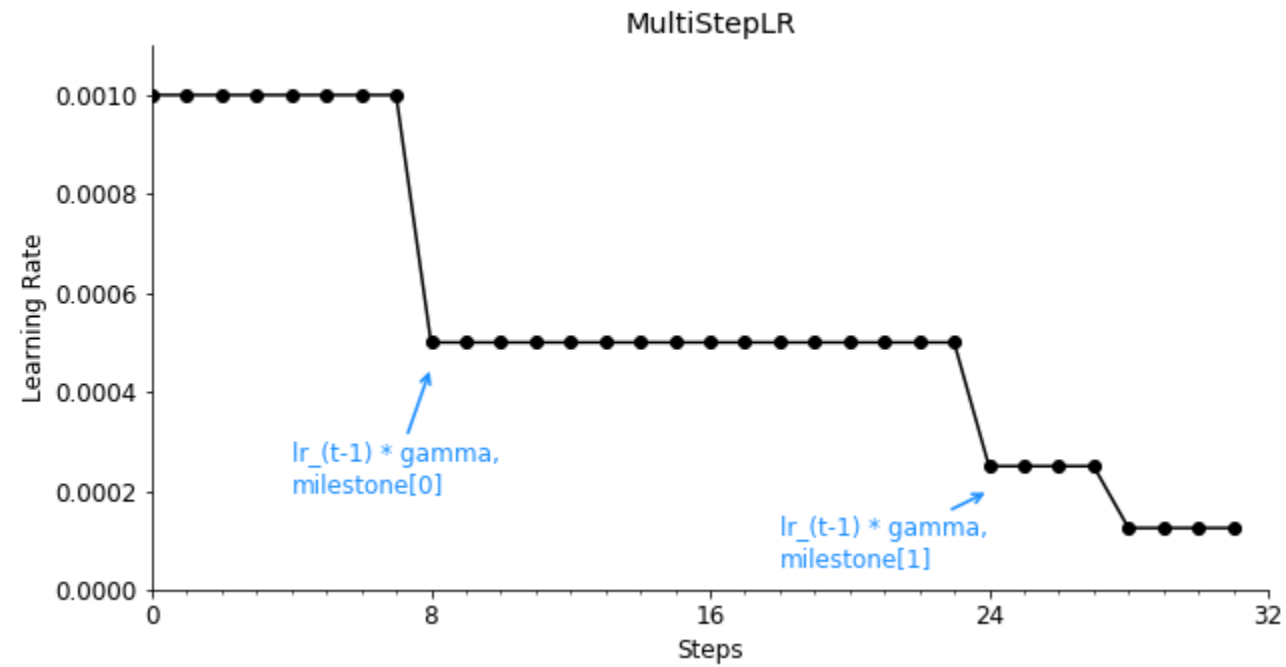
NEED FOR LEARNING RATE SCHEDULERS



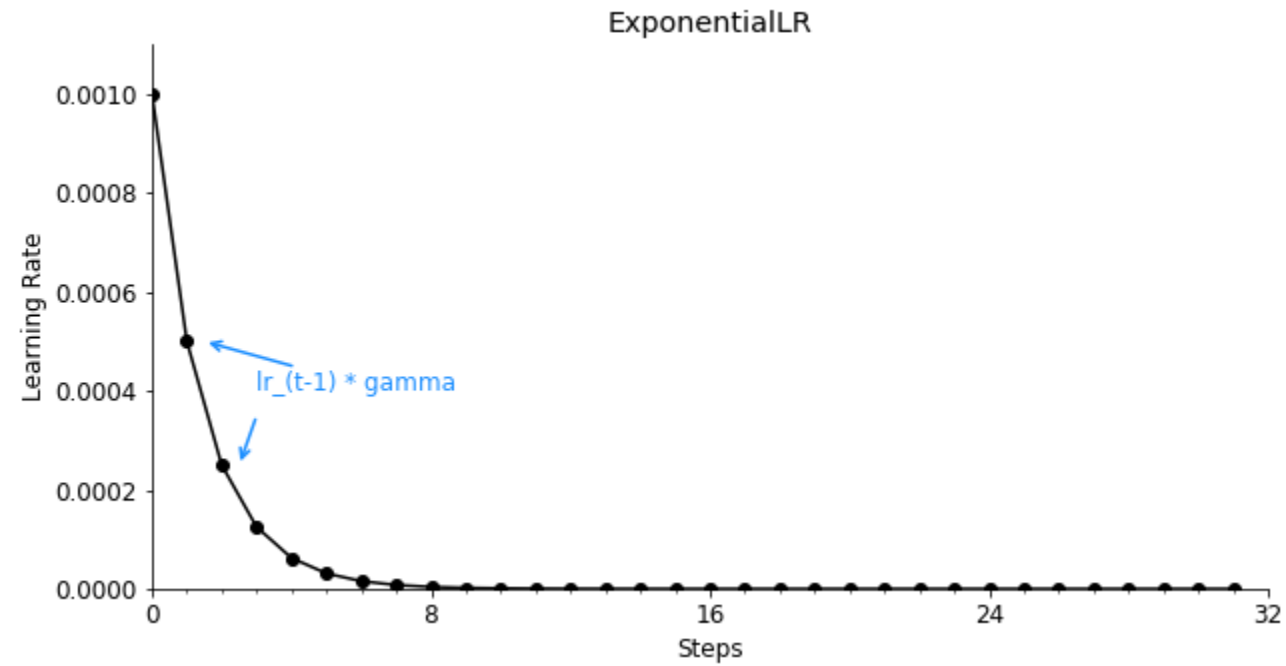
STEP-LR



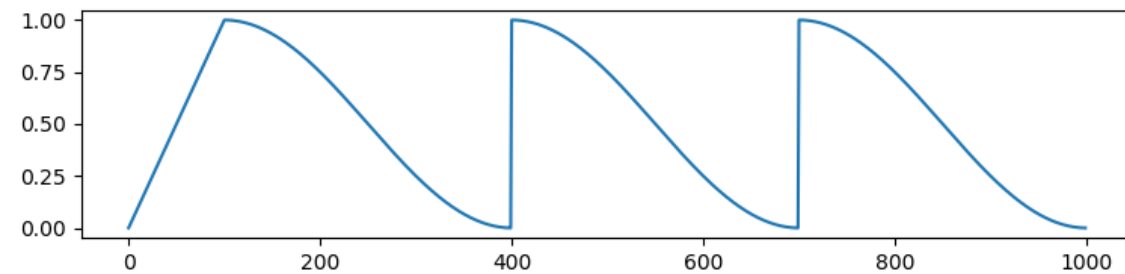
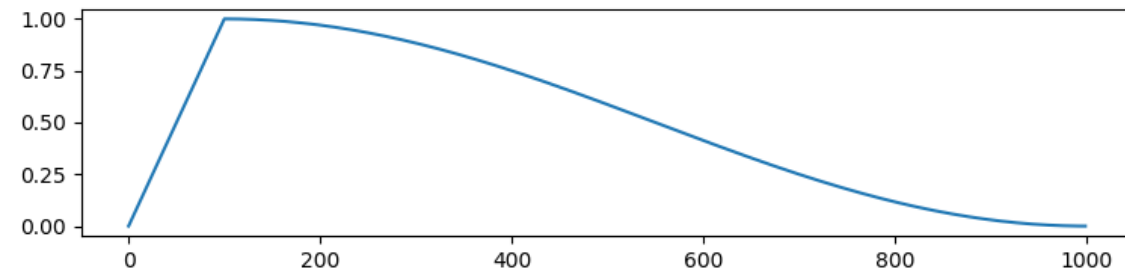
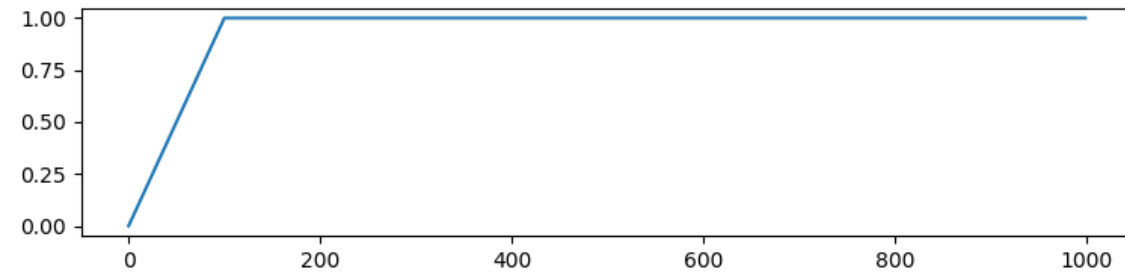
MULTI-STEP-LR



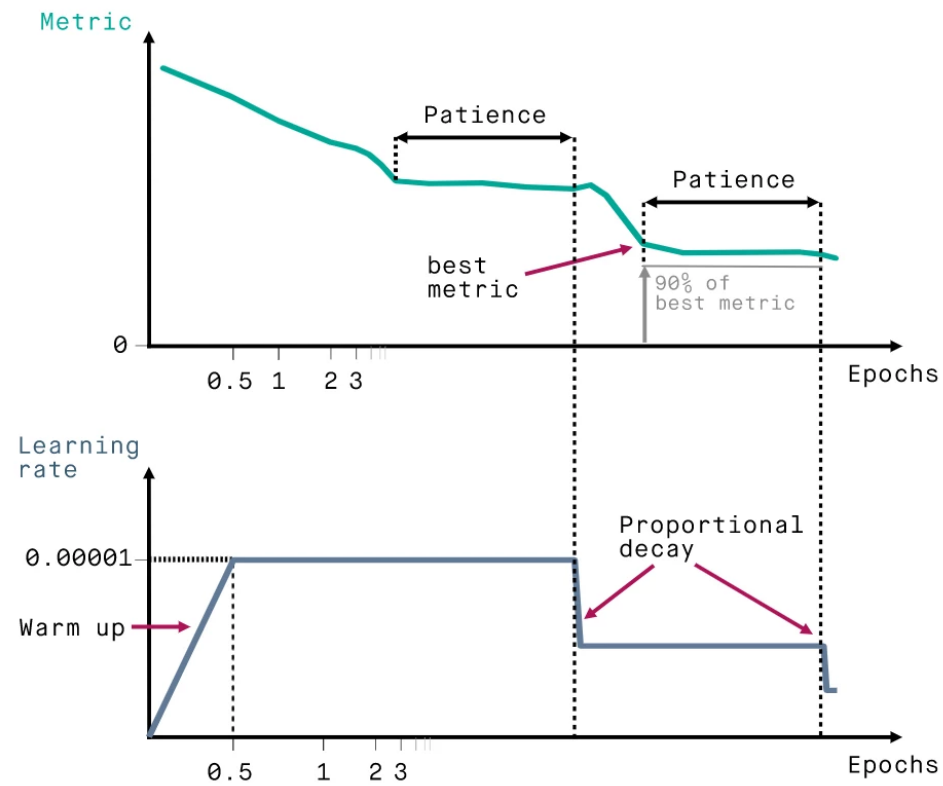
EXPONENTIAL-LR



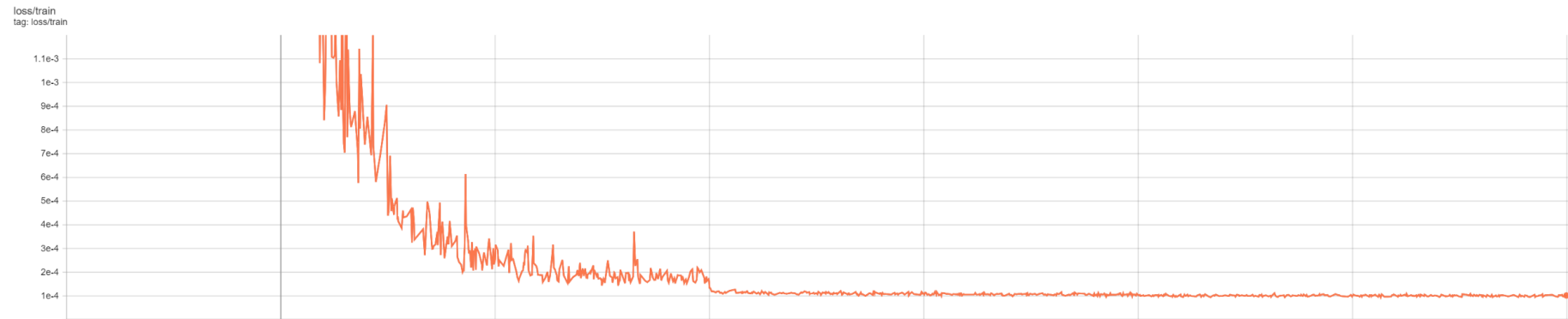
WARMUP



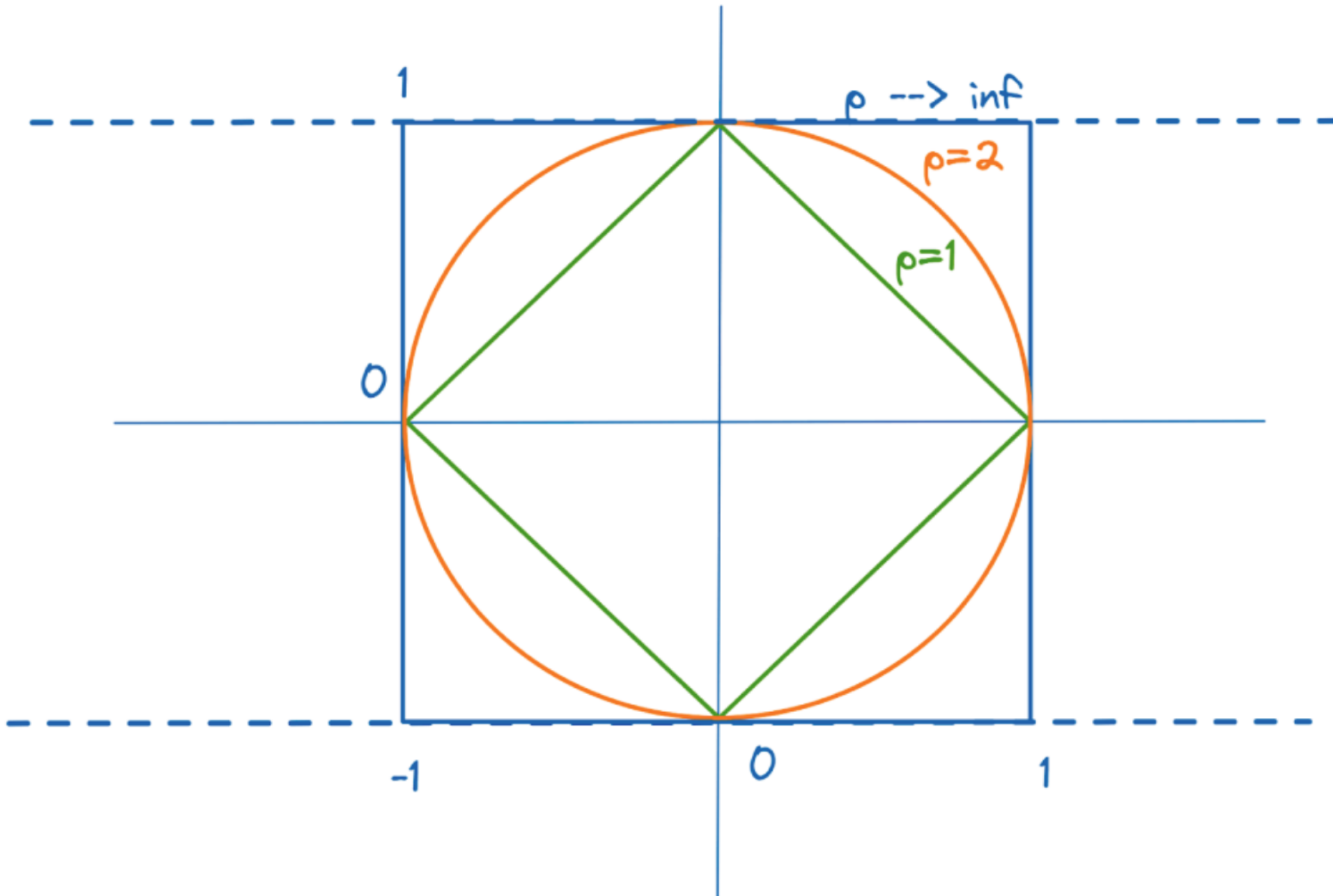
REDUCE-LR-ON-PLATEAU



EFFECT ON PERFORMANCE



9 - REGULARIZATION



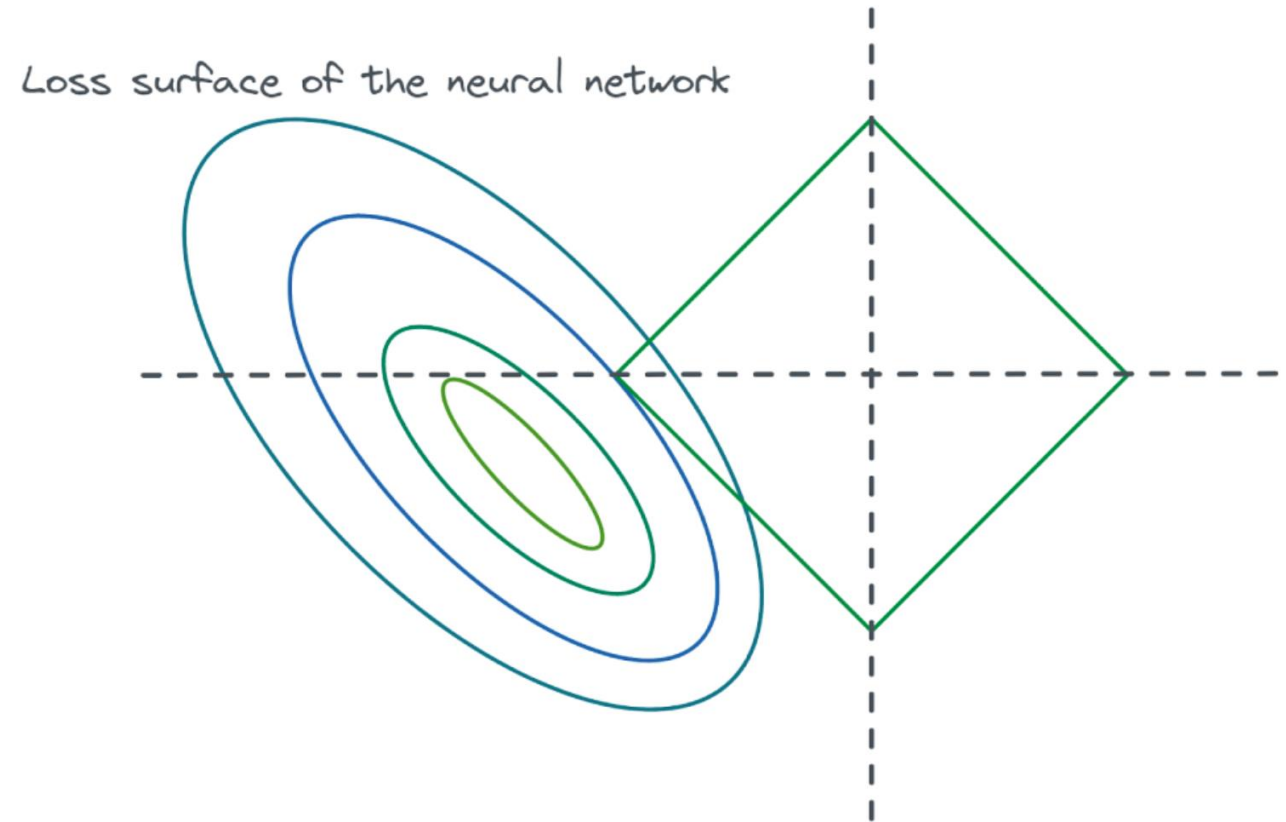
L1 REGULARIZATION

$$L_1(\mathbf{x}) = ||\mathbf{x}||_1 = \sum_{i=1}^n |x_i|$$

$$\tilde{\mathcal{L}}(\mathbf{w}) = \mathcal{L}(\mathbf{w}) + \alpha ||\mathbf{w}||_1$$



L1 REGULARIZATION



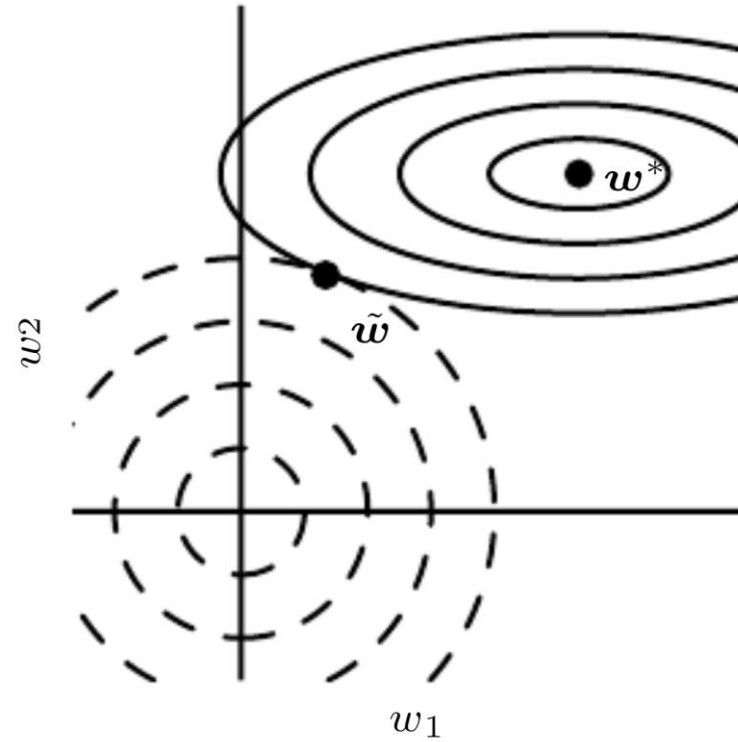
L2 REGULARIZATION

$$L_2(\mathbf{x}) = ||\mathbf{x}||_2 = \left(\sum_{i=1}^n |x_i|^2 \right)^{1/2}$$

$$\tilde{\mathcal{L}}(\mathbf{w}) = \mathcal{L}(\mathbf{w}) + \frac{\alpha}{2} ||\mathbf{w}||_2^2$$



L2 REGULARIZATION



L2 REGULARIZATION - WEIGHT DECAY

The proof for this well-known fact is straight-forward. SGD without weight decay has the following iterates on $f_t^{\text{reg}}(\boldsymbol{\theta}) = f_t(\boldsymbol{\theta}) + \frac{\lambda'}{2} \|\boldsymbol{\theta}\|_2^2$:

$$\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \alpha \nabla f_t^{\text{reg}}(\boldsymbol{\theta}_t) = \boldsymbol{\theta}_t - \alpha \nabla f_t(\boldsymbol{\theta}_t) - \alpha \lambda' \boldsymbol{\theta}_t. \quad (5)$$

SGD with weight decay has the following iterates on $f_t(\boldsymbol{\theta})$:

$$\boldsymbol{\theta}_{t+1} \leftarrow (1 - \lambda) \boldsymbol{\theta}_t - \alpha \nabla f_t(\boldsymbol{\theta}_t). \quad (6)$$

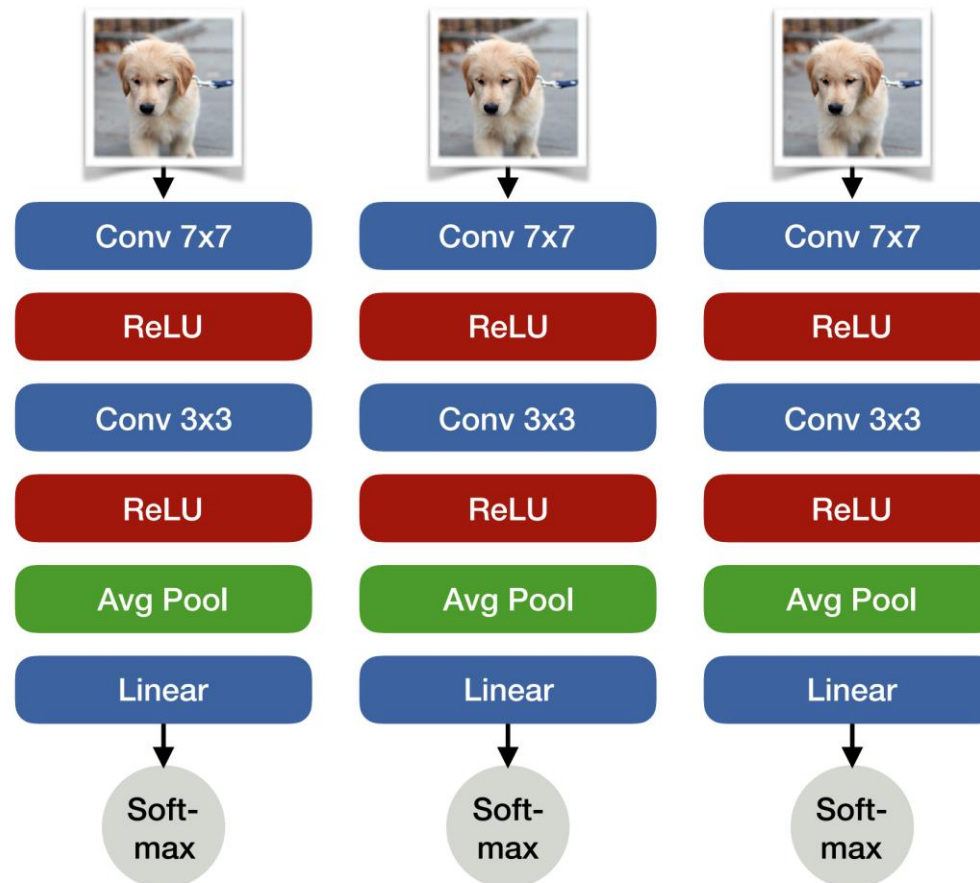
These iterates are identical since $\lambda' = \frac{\lambda}{\alpha}$.

□

10 - ENSEMBLES

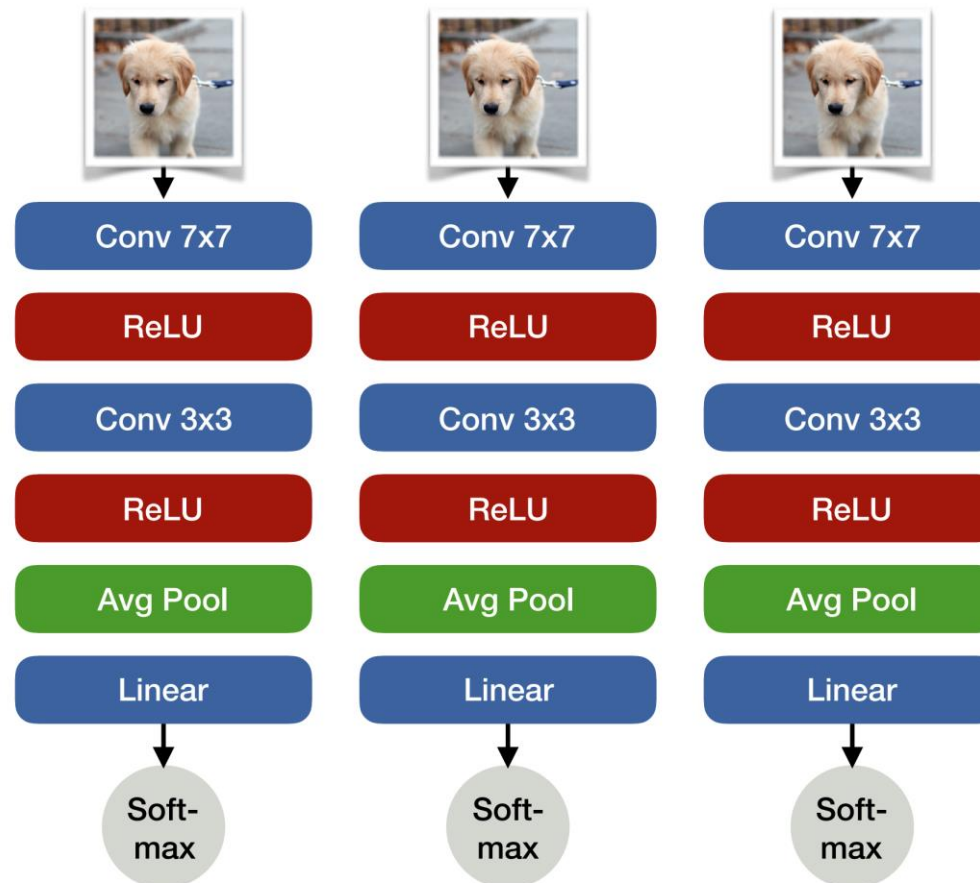
INTRODUCTION

- Train multiple models
- Average the predictions of multiple models



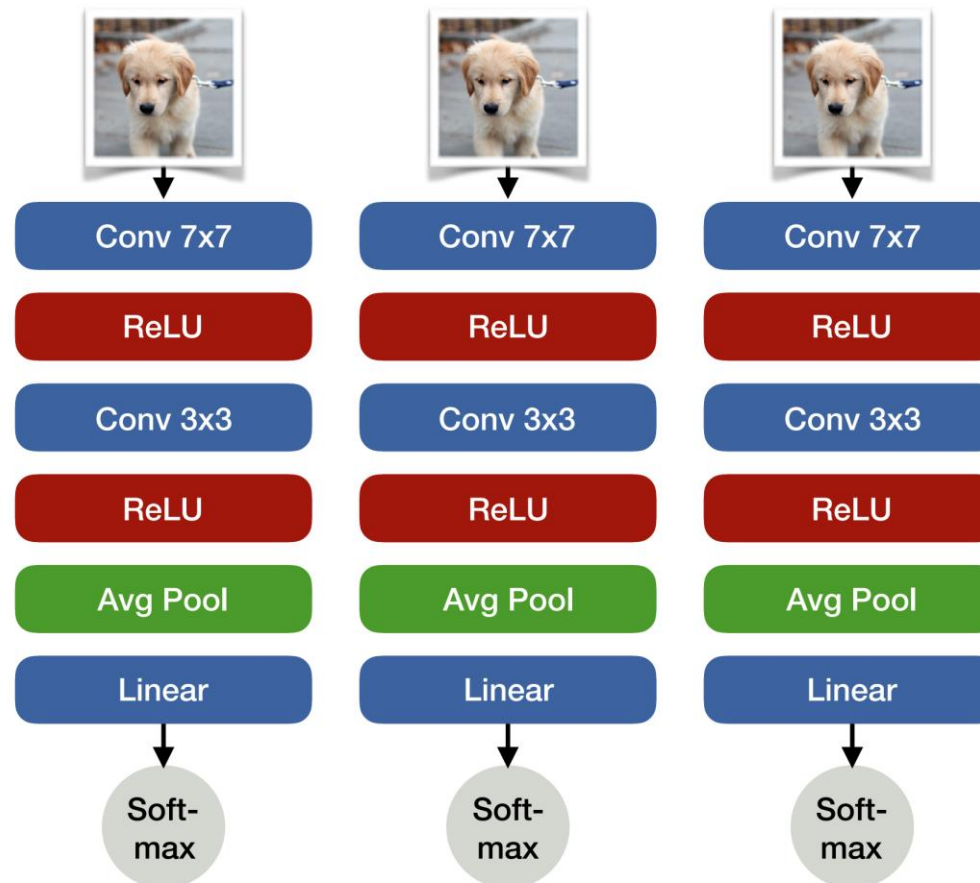
EVOLUTION

- Pre deep learning:
 - Use different subsets of training data
- Deep Learning:
 - Use different random initializations / data augmentation
 - Different local minima



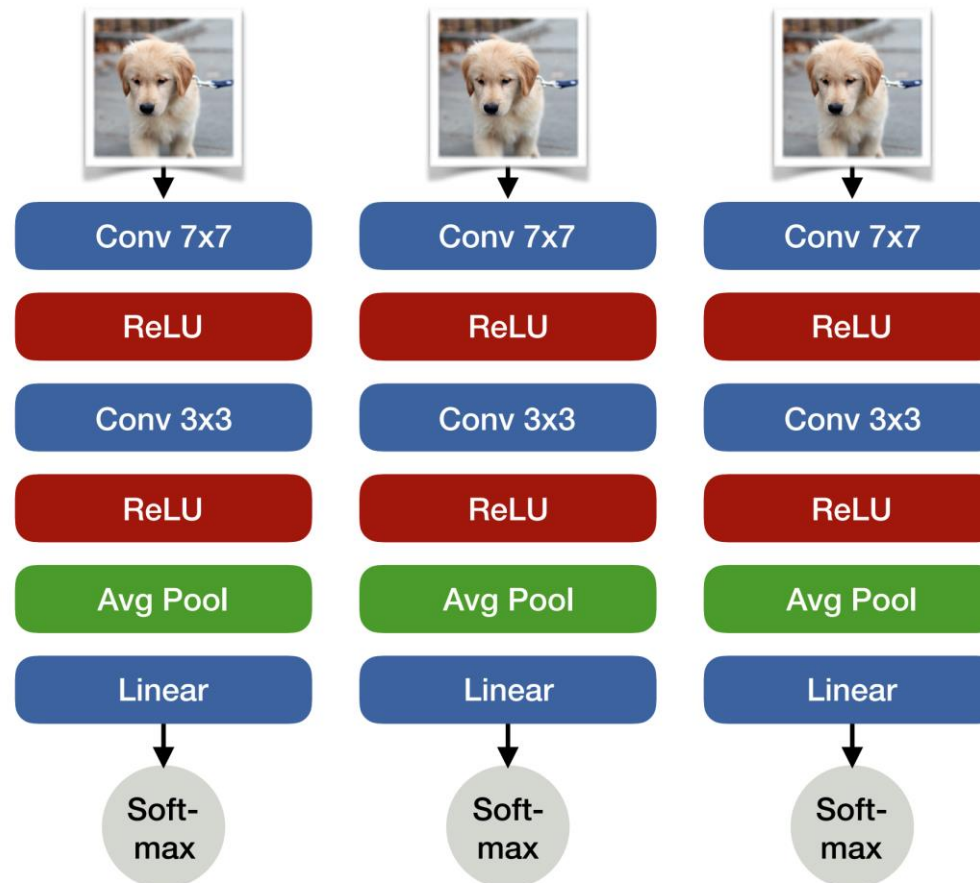
DO THEY WORK?

- Fewer parameters per model
- Each model overfits in its own way
- Usually 1-3% boost in accuracy
- Longer training



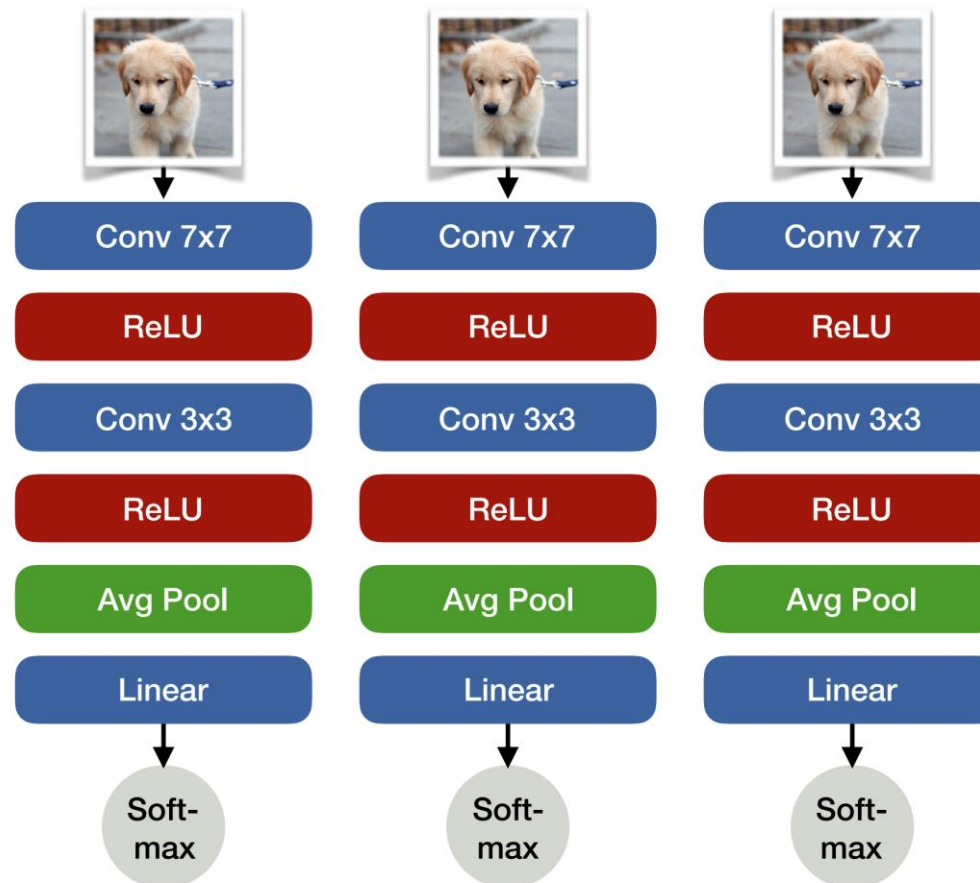
DO THEY WORK?

- For a convex loss function
- Loss of average prediction < loss of individual models



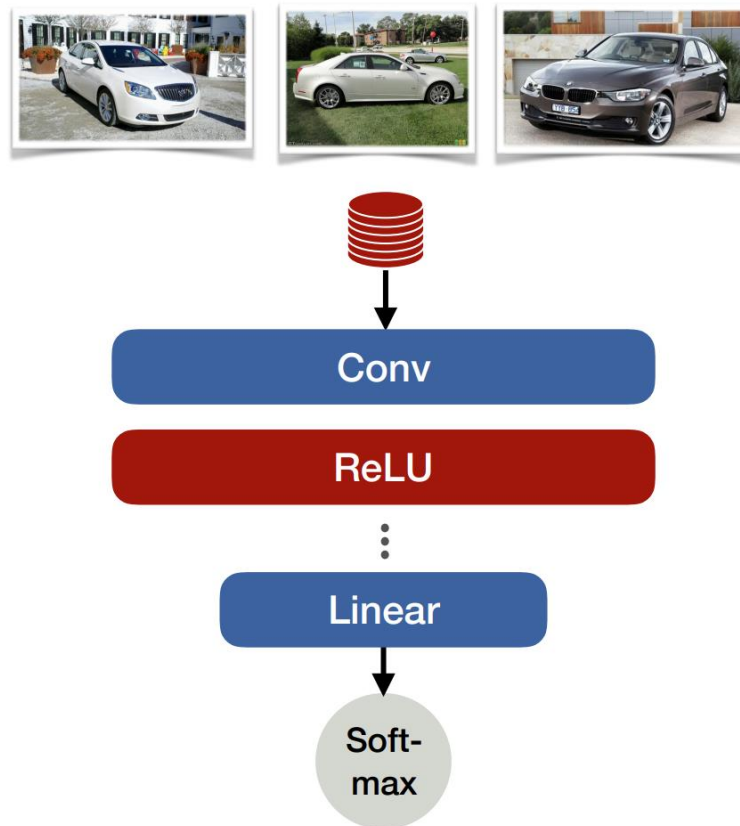
WHEN TO USE THEM

- If you have the compute power
- If you need the last bit of accuracy

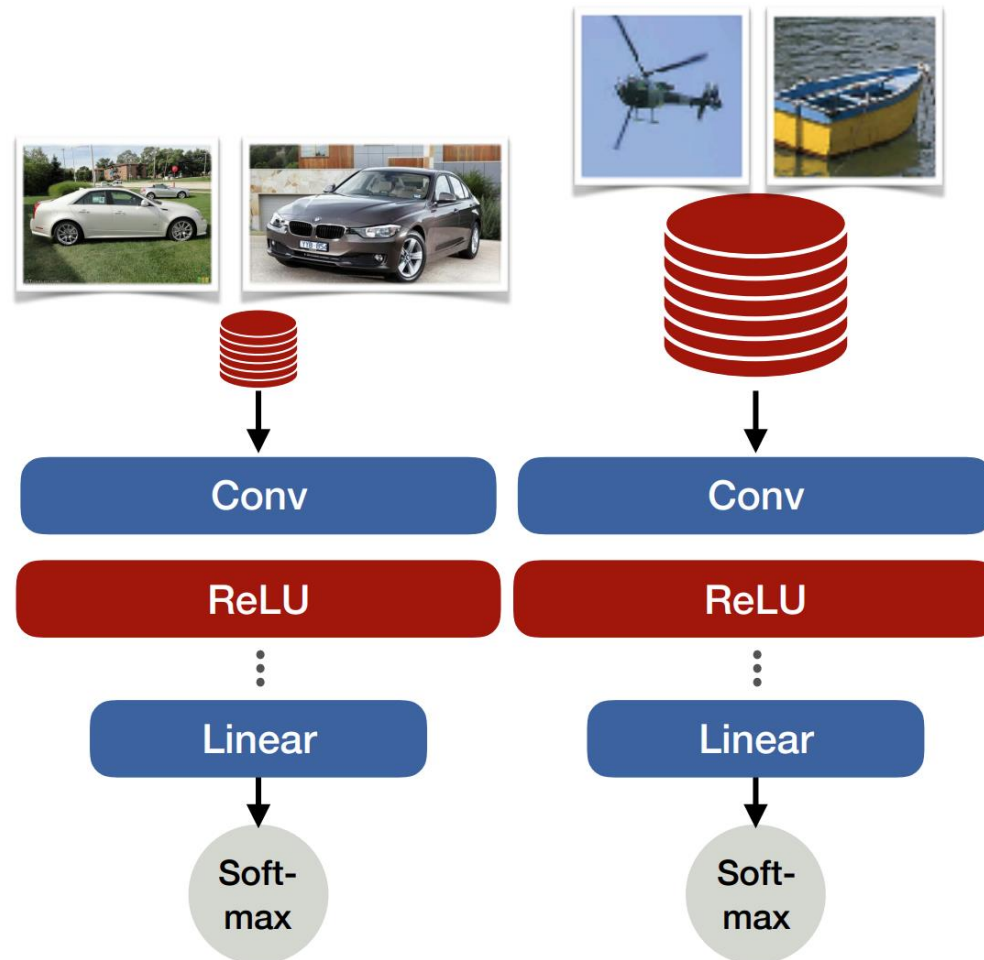


1 1 - TRANSFER LEARNING

HOW TO TRAIN LARGE MODELS ON SMALL DATASETS?



PRETRAINING / FINE TUNING



LARGE DATASETS TO PRETRAIN



WIKIPEDIA
The Free Encyclopedia



REFERENCES

- <https://www.blog.dailydoseofds.com>
- <https://www.deeplearningbook.org>
- <https://www.philkr.net>
- <https://www.deeplearningwizard.com>
- <https://huggingface.co>
- <https://towardsdatascience.com/a-visual-guide-to-learning-rate-schedulers-in-pytorch-24bbb262c863>
- <https://wiki.cloudfactory.com/docs/mp-wiki/scheduler/reducelronplateau>
- <https://www.youtube.com/@deepbean>
- <https://www.pinecone.io/learn/regularization-in-neural-networks/>

